

Grado en Ingeniería Electrónica de Comunicaciones
Procesamiento de Señales · Curso 2026-2027

Apuntes

Tema 4. Filtros

Jesús Chacón Sombría

Cómo usar estos apuntes

Estos apuntes desarrollan el contenido de las cinco presentaciones del tema: la sección 1 corresponde a la parte 1 (filtros ideales, especificaciones y fase), la sección 2 a la parte 2 (filtros continuos), la sección 3 a la parte 3 (filtros IIR), la sección 4 a la parte 4 (filtros FIR) y la sección 5 a la parte 5 (fase cero, fase mínima, paso todo y peine). Los ejemplos de código están en `teoría/T4/codigo/`, en Python (`scipy.signal`) y en Matlab, y se pueden ejecutar tal cual. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

Índice general

1 Filtros ideales, especificaciones y fase 2
- 1.1 Filtros ideales . . . 2
- 1.2 Especificaciones . . . 3
- 1.3 Fase y retardo de grupo . . . 3
- 1.4 Polos, ceros y diseño a mano . . . 4
- 1.5 Filtros FIR e IIR . . . 5

2 Filtros continuos 6
- 2.1 Aproximación del módulo . . . 6
- 2.2 Butterworth . . . 6
- 2.3 Chebyshev . . . 8
- 2.4 Elíptico . . . 9
- 2.5 Comparación . . . 9
- 2.6 Transformaciones de frecuencia . . . 10

3 Filtros IIR 13
- 3.1 Invarianza al impulso . . . 13
- 3.2 Transformada bilineal . . . 13
- 3.3 Procedimiento de diseño . . . 14
- 3.4 Transformaciones de frecuencia en discreto . . . 17
- 3.5 Implementación . . . 17

4 Filtros FIR 17
- 4.1 Fase lineal . . . 18
- 4.2 Método de las ventanas . . . 19
- 4.3 Fenómeno de Gibbs y ventanas fijas . . . 20
- 4.4 Ventana de Kaiser . . . 21
- 4.5 Parks–McClellan y muestreo en frecuencia . . . 23
- 4.6 Coste y retardo . . . 23

5 Filtros especiales **24**
5.1 Fase cero 24
5.2 Fase mínima, máxima y mixta 25
5.3 Filtros paso todo 26
5.4 Filtros en peine 26

Bibliografía **27**

1 Filtros ideales, especificaciones y fase

Un *filtro* es un sistema LTI estable que modifica selectivamente las componentes en frecuencia de una señal. Lo describen su respuesta al impulso o su respuesta en frecuencia,

$$h(t) \overset{\mathscr{F}}{\longleftrightarrow} H(j\omega), \qquad h[n] \overset{\mathscr{F}}{\longleftrightarrow} H(\mathrm{e}^{j\Omega}),$$

y, como vimos en el Tema 1, una sinusoide de frecuencia ω_0 sale de él multiplicada por $|H(j\omega_0)|$ y desfasada $\arg H(j\omega_0)$. Seguimos la notación del curso: ω (rad/s) es la frecuencia de las señales continuas y Ω (rad/muestra) la de las discretas; con muestreo a f_s, $\Omega = \omega/f_s = 2\pi f/f_s$. La respuesta de un filtro discreto es periódica de periodo 2π, así que basta describirla en $[-\pi, \pi]$, y si $h[n]$ es real, en $[0, \pi]$, porque $|H|$ es par y $\arg H$ impar.

1.1 Filtros ideales

Los cuatro filtros selectivos ideales (figura 1) dejan pasar sin cambios una banda de frecuencias y anulan el resto:

$$H_{\mathrm{PB}}(\mathrm{e}^{j\Omega}) = \begin{cases} 1 & |\Omega| < \Omega_c \\ 0 & \Omega_c < |\Omega| \le \pi \end{cases}, \qquad H_{\mathrm{BP}}(\mathrm{e}^{j\Omega}) = \begin{cases} 1 & \Omega_1 < |\Omega| < \Omega_2 \\ 0 & \text{en el resto de } [-\pi, \pi] \end{cases},$$

con el paso alto $H_{\mathrm{PA}} = 1 - H_{\mathrm{PB}}$ y el supresor de banda $H_{\mathrm{SB}} = 1 - H_{\mathrm{BP}}$. En continuo se definen igual con ω en lugar de Ω. El valor absoluto es imprescindible: un filtro con respuesta al impulso real actúa igual sobre ω y $-\omega$.

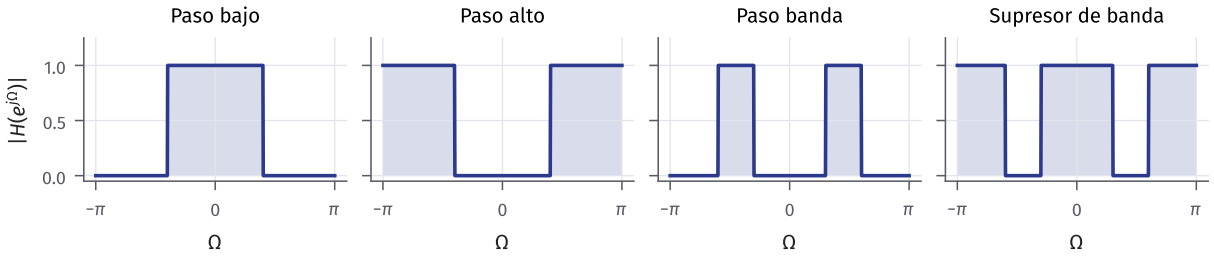


Figura 1. Los cuatro filtros ideales en discreto.

Erratas de versiones anteriores

En las transparencias antiguas el paso banda y el supresor ideales se escribían como $\omega_{c1} < \omega < \omega_{c2}$, sin valor absoluto, y el paso alto de la parte FIR como «1 si $|\omega| < \omega_c$». Las definiciones correctas son las de arriba.

Ningún filtro ideal se puede construir. La respuesta al impulso del paso bajo ideal es

$$h[n] = \frac{1}{2\pi}\int_{-\Omega_c}^{\Omega_c} \mathrm{e}^{j\Omega n}\,\mathrm{d}\Omega = \frac{\sin(\Omega_c n)}{\pi n}$$

(con $h[0] = \Omega_c/\pi$), que es distinta de cero para $n < 0$, de modo que el sistema no es causal, y tiene duración infinita. En continuo ocurre lo mismo con $h(t) = \sin(\omega_c t)/(\pi t)$. Diseñar un filtro consiste en *aproximar* el ideal con un sistema causal, estable y de orden finito, admitiendo una tolerancia.

1.2 Especificaciones

La tolerancia se expresa con una *plantilla* (figura 2). Para un paso bajo:

- en la banda de paso, $0 \le \omega \le \omega_p$, la atenuación no supera α_p dB: $1/E \le |H| \le 1$ con $\alpha_p = 20\log_{10} E$;
- en la banda de supresión, $\omega \ge \omega_s$, la atenuación es al menos α_s dB: $|H| \le 1/A$ con $\alpha_s = 20\log_{10} A$;
- en la banda de transición, $\omega_p < \omega < \omega_s$, no se impone nada.

La *frecuencia de corte* ω_c es aquella en la que $|H(j\omega_c)| = 1/\sqrt{2}$, es decir, la atenuación es de 3 dB. Dos cocientes resumen la dificultad del diseño: la *selectividad* $k = \omega_p/\omega_s < 1$, más exigente cuanto más se acerca a 1, y la *discriminación*

$$k_1 = \frac{\varepsilon}{\sqrt{A^2-1}}, \qquad \varepsilon = \sqrt{E^2-1},$$

más exigente cuanto más pequeña. En los filtros FIR es más habitual dar los rizados: $1-\delta_p \le |H| \le 1+\delta_p$ en la banda de paso y $|H| \le \delta_s$ en la de supresión, con $\alpha_s = -20\log_{10}\delta_s$. Para los demás tipos de filtro hay un par de bordes por cada transición ($\omega_{p1}, \omega_{p2}, \omega_{s1}, \omega_{s2}$ en un paso banda).

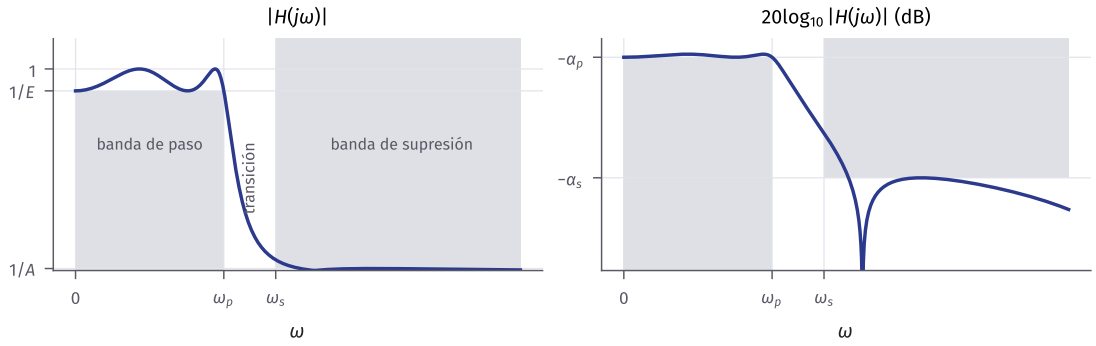


Figura 2. Plantilla de un paso bajo en escala lineal y en decibelios, con un filtro elíptico que la cumple. Las zonas sombreadas están prohibidas.

1.3 Fase y retardo de grupo

Un sistema que no distorsiona solo escala y retrasa: $y[n] = K\,x[n-n_0]$, con $H(\mathrm{e}^{j\Omega}) = K\mathrm{e}^{-j\Omega n_0}$. Su fase es *lineal* en Ω. En general se define el *retardo de grupo*

$$\tau(\Omega) = -\frac{\mathrm{d}\arg H(\mathrm{e}^{j\Omega})}{\mathrm{d}\Omega} \quad \text{(muestras)},$$

que mide cuánto se retrasa la envolvente de un paquete de frecuencias en torno a Ω. Si τ es constante, todas las componentes se retrasan lo mismo y la forma de onda se conserva; si no, se produce *distorsión*

de fase. La figura 3 lo ilustra con una onda cuadrada: un IIR y un FIR con módulos parecidos dan salidas muy distintas, porque el IIR retrasa más los armónicos cercanos al corte. La distorsión de fase importa en transmisión de datos, en electrocardiografía o en audio de alta calidad; el oído es poco sensible a ella en régimen permanente, pero no en los transitorios.

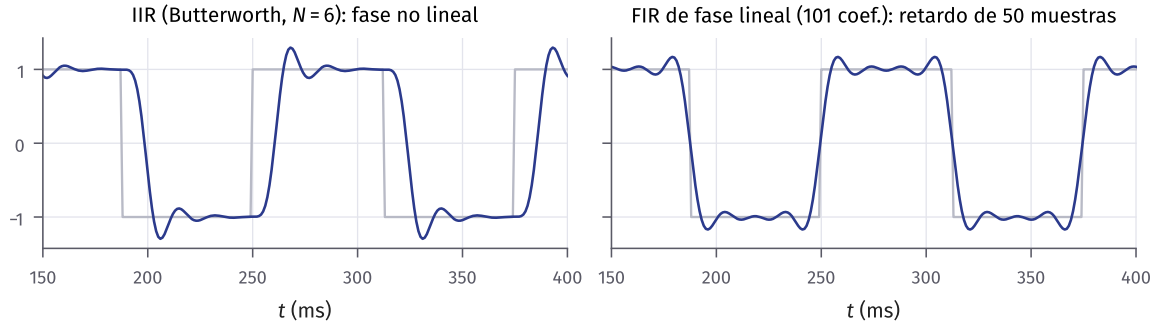


Figura 3. Onda cuadrada de 8 Hz ($f_s = 1$ kHz) filtrada con un Butterworth de orden 6 y con un FIR de fase lineal de 101 coeficientes, ambos con corte en 60 Hz. La salida del FIR se ha adelantado 50 muestras para compararla.

1.4 Polos, ceros y diseño a mano

La respuesta en frecuencia de un sistema racional queda determinada por la posición de sus polos y ceros, que conocemos del Tema 1 (figura 4):

- un *polo real* en $-a$, $H(s) = a/(s+a)$, atenúa a partir de $\omega \approx a$ con una pendiente de -20 dB/década, y cada polo adicional añade otros -20 dB/década;
- un *par de polos* $-\sigma \pm j\omega_a$ produce una resonancia cerca de ω_a, tanto más aguda cuanto menor es σ. Con $\sigma = 0$, $H(s) = \omega_n^2/(s^2 + \omega_n^2)$ tiene los polos sobre el eje imaginario: es *marginalmente estable*, su respuesta al impulso es una sinusoide que no se amortigua y no sirve como filtro;
- un *par de ceros* en $\pm j\omega_n$ anula exactamente la frecuencia ω_n. Sin polos, $|H|$ crecería como ω^2; con dos polos con parte real negativa el sistema es propio (tantos polos como ceros, condición para que sea realizable) y la ganancia vuelve a 1 lejos de ω_n.

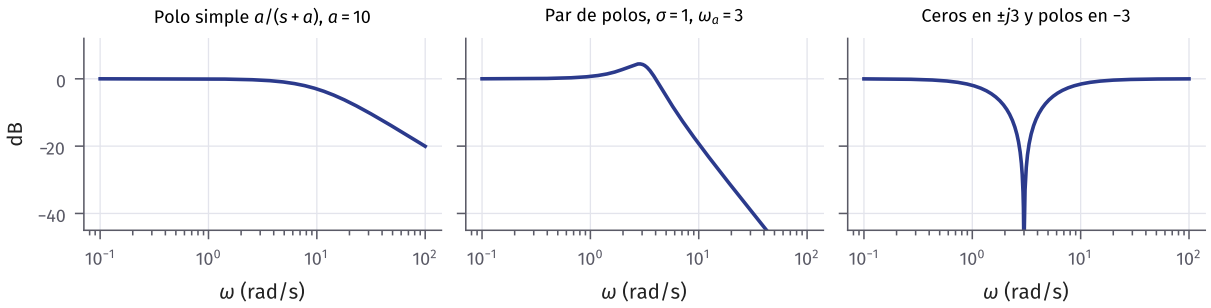


Figura 4. Efecto de un polo real, de un par de polos complejos y de un par de ceros imaginarios compensado con dos polos.

Erratas de versiones anteriores

Las transparencias antiguas decían que $\omega_n^2/(s^2 + \omega_n^2)$ «es inestable para $\omega = \omega_n$»: el sistema es marginalmente estable (lo es o no para todas las frecuencias a la vez) y, en todo caso, queda fuera

de los filtros que estudiamos, que son estables. También mezclaban w_a y ω_a en la misma expresión.

En discreto se razona igual con la circunferencia unidad en lugar del eje $j\omega$. Un *filtro de muesca* que elimine Ω_0 tiene ceros en $\mathrm{e}^{\pm j\Omega_0}$ y polos en el mismo ángulo con radio $r < 1$:

$$H(z) = \frac{(1 - \mathrm{e}^{j\Omega_0} z^{-1})(1 - \mathrm{e}^{-j\Omega_0} z^{-1})}{(1 - r\mathrm{e}^{j\Omega_0} z^{-1})(1 - r\mathrm{e}^{-j\Omega_0} z^{-1})} = \frac{1 - 2\cos\Omega_0\, z^{-1} + z^{-2}}{1 - 2r\cos\Omega_0\, z^{-1} + r^2 z^{-2}}.$$

Lejos de Ω_0 cada polo está casi a la misma distancia del punto $\mathrm{e}^{j\Omega}$ que su cero, y $|H| \approx 1$; la anchura de la muesca es aproximadamente $2(1 - r)$ rad (figura 5). Este diseño a mano funciona para filtros muy sencillos, pero no para filtros selectivos con especificaciones exigentes: para ellos hacen falta los métodos sistemáticos de las secciones siguientes.

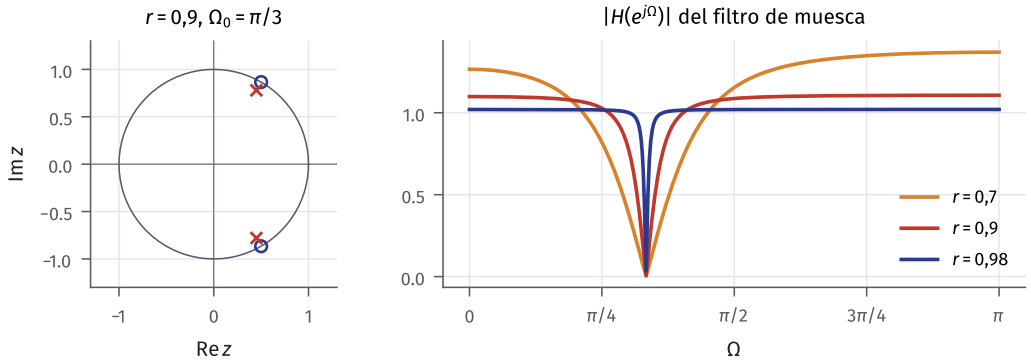


Figura 5. Filtro de muesca en $\Omega_0 = \pi/3$: diagrama de polos y ceros y respuesta para tres radios.

1.5 Filtros FIR e IIR

Un filtro discreto causal de orden finito tiene función de transferencia y ecuación en diferencias

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}}, \qquad y[n] = \sum_{k=0}^{M} b_k\, x[n-k] - \sum_{k=1}^{N} a_k\, y[n-k].$$

Si algún $a_k \neq 0$ hay realimentación y la respuesta al impulso es infinita: filtro *IIR*. Si todos los a_k son nulos, $h[n] = b_n$ tiene $M + 1$ muestras: filtro *FIR*. La cuadro 1 resume sus diferencias, que guían la elección.

Cuadro 1. Comparación entre filtros IIR y FIR.

	IIR	FIR
Diseño	A partir de un filtro continuo	Directo en discreto (ventanas, optimización)
Orden	Bajo	Alto: decenas o cientos de coeficientes
Fase	No lineal	Puede ser exactamente lineal
Estabilidad	Hay que comprobarla	Siempre estables
Implementación	Sensible a la cuantificación; secciones de orden 2	Robusta; admite FFT para órdenes altos

Ejercicios 1–3. *Herramienta interactiva:* `herramientas/server/T4_Filtros_Discretos_PZMap.py`.

2 Filtros continuos

2.1 Aproximación del módulo

Diseñamos un paso bajo continuo, el *prototipo*, y obtendremos los demás tipos con transformaciones de frecuencia (apartado 2.6). Como $|H(j\omega)|^2$ es una función par de ω, se busca de la forma

$$|H(j\omega)|^2 = \frac{1}{1+P(\omega^2)},$$

con P pequeño en la banda de paso y grande en la de supresión. Cada familia de filtros corresponde a una elección de P. Conocido $|H(j\omega)|^2 = H(s)H(-s)\big|_{s=j\omega}$, se calculan las raíces de $H(s)H(-s)$, que aparecen en parejas simétricas respecto al eje imaginario, y se asignan a $H(s)$ las del semiplano izquierdo: el filtro resulta estable.

Erratas de versiones anteriores

Las transparencias antiguas escribían $|H(j\omega)| = 1/(1+P(\omega))$. Lo que se diseña es el cuadrado del módulo y P depende de ω^2; con la forma antigua no salen las fórmulas de Butterworth y Chebyshev que siguen.

Los datos del diseño son ω_p, ω_s, α_p y α_s, y de ellos se derivan

$$E = 10^{\alpha_p/20}, \quad A = 10^{\alpha_s/20}, \quad \varepsilon = \sqrt{E^2-1}, \quad k = \frac{\omega_p}{\omega_s}, \quad k_1 = \frac{\varepsilon}{\sqrt{A^2-1}}.$$

Usaremos como ejemplo común un paso bajo con $\alpha_p = 1\,\text{dB}$ hasta $1\,\text{kHz}$ y $\alpha_s = 40\,\text{dB}$ desde $5\,\text{kHz}$: $\omega_p = 2\pi\cdot 1000\,\text{rad/s}$, $\omega_s = 2\pi\cdot 5000\,\text{rad/s}$, $E = 1{,}122$, $A = 100$, $\varepsilon = 0{,}5088$, $k = 0{,}2$ y $k_1 = 0{,}00509$.

2.2 Butterworth

Definición 2.1: Filtro de Butterworth

$$|H(j\omega)|^2 = \frac{1}{1+(\omega/\omega_c)^{2N}}$$

Es *máximamente plano*: las $2N-1$ primeras derivadas de $|H|^2$ se anulan en $\omega = 0$. Es monótono, vale $1/\sqrt{2}$ en ω_c sea cual sea el orden y cae $20N$ dB/década. Las raíces de $1+(s/j\omega_c)^{2N} = 0$ están sobre una circunferencia de radio ω_c; las del semiplano izquierdo son los polos

$$p_k = \omega_c\, e^{j\pi\frac{2k+N-1}{2N}}, \qquad k = 1,\dots,N,$$

separados un ángulo π/N (figura 6).

Para obtener el orden se imponen los dos bordes de la plantilla:

$$\frac{1}{1+(\omega_p/\omega_c)^{2N}} \geq \frac{1}{E^2} \;\Leftrightarrow\; \left(\frac{\omega_p}{\omega_c}\right)^{2N} \leq E^2-1, \qquad \frac{1}{1+(\omega_s/\omega_c)^{2N}} \leq \frac{1}{A^2} \;\Leftrightarrow\; \left(\frac{\omega_s}{\omega_c}\right)^{2N} \geq A^2-1.$$

Dividiendo una entre otra se elimina ω_c.

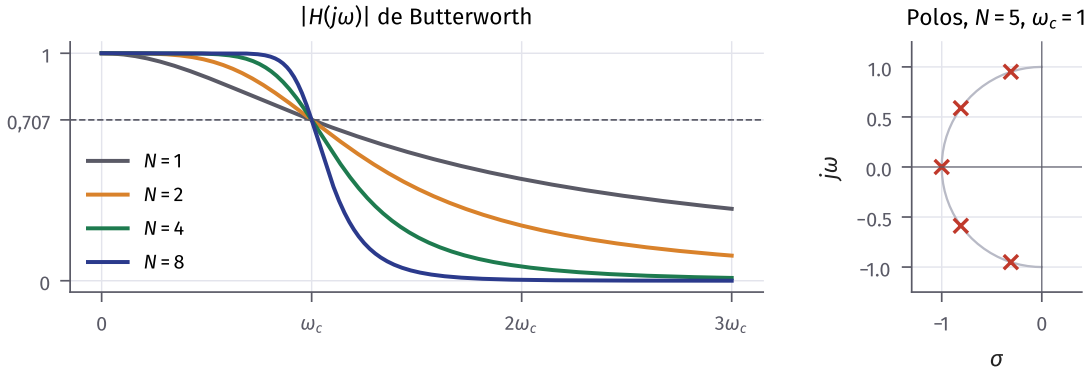


Figura 6. Módulo del filtro de Butterworth para varios órdenes y polos para $N = 5$.

Propiedad 2.2: Orden y corte de un Butterworth

$$N \geq \frac{\log_{10} \dfrac{A^2-1}{E^2-1}}{2\log_{10} \dfrac{\omega_s}{\omega_p}} = \frac{\log(1/k_1)}{\log(1/k)}, \qquad \frac{\omega_p}{(E^2-1)^{1/2N}} \leq \omega_c \leq \frac{\omega_s}{(A^2-1)^{1/2N}}.$$

Al redondear N hacia arriba queda margen y ω_c se puede elegir en un intervalo: en el extremo izquierdo se cumple exactamente la banda de paso; en el derecho, la de supresión. `buttord` de Matlab y de `scipy.signal` devuelve el izquierdo.

Ejemplo 2.3: Butterworth

Con las especificaciones comunes, $E^2 - 1 = 0{,}2589$ y $A^2 - 1 = 9999$:

$$N \geq \frac{\log_{10}(9999/0{,}2589)}{2\log_{10} 5} = \frac{4{,}587}{1{,}398} = 3{,}28 \;\rightarrow\; N = 4,$$

$$\omega_c \in \left[\frac{6283}{0{,}2589^{1/8}},\ \frac{31416}{9999^{1/8}}\right] = [7439,\ 9935]\ \text{rad/s},$$

es decir, f_c entre 1184 y 1581 Hz.

En código

Orden, corte y respuesta del Butterworth del ejemplo 2.3.

Python

```python
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt

wp = 2*np.pi*1e3; ws = 2*np.pi*5e3  # rad/s
ap, as_ = 1, 40                     # dB
N, wc = signal.buttord(wp, ws, ap, as_,
                       analog=True)
print(N, wc)                        # 4, 7439
b, a = signal.butter(N, wc, analog=True)
w = np.linspace(0, 2*np.pi*8e3, 1000)
w, H = signal.freqs(b, a, w)
plt.plot(w/(2*np.pi), 20*np.log10(abs(H)))
plt.xlabel("f (Hz)"); plt.ylabel("dB")
plt.show()
```

Matlab

```matlab
wp = 2*pi*1e3; ws = 2*pi*5e3;       % rad/s
ap = 1; as = 40;                    % dB
[N, wc] = buttord(wp, ws, ap, as, 's');
disp([N, wc])                       % 4, 7439
[b, a] = butter(N, wc, 's');
w = linspace(0, 2*pi*8e3, 1000);
H = freqs(b, a, w);
plot(w/(2*pi), 20*log10(abs(H)))
xlabel('f (Hz)'); ylabel('dB')
```

2.3 Chebyshev

Los *polinomios de Chebyshev* se definen por

$$T_N(x) = \begin{cases} \cos(N \arccos x) & |x| \le 1 \\ \cosh(N \operatorname{arccosh} x) & |x| > 1 \end{cases}, \qquad T_0 = 1,\ T_1 = x,\ T_{N+1} = 2xT_N - T_{N-1}.$$

En $[-1, 1]$ oscilan entre -1 y 1, con N ceros; fuera crecen como $2^{N-1}x^N$, mucho más deprisa que x^N.

Definición 2.4: Filtros de Chebyshev

$$\text{Tipo I: } |H(j\omega)|^2 = \frac{1}{1+\varepsilon^2 T_N^2(\omega/\omega_p)}, \qquad \text{Tipo II: } |H(j\omega)|^2 = \frac{\varepsilon_2^2\, T_N^2(\omega_s/\omega)}{1+\varepsilon_2^2\, T_N^2(\omega_s/\omega)},\ \varepsilon_2^2 = A^2 - 1.$$

El tipo I tiene rizado entre 1 y $1/E$ en la banda de paso (equirrizado) y cae monótonamente en la de supresión; sus polos están sobre una elipse. El tipo II, o inverso, es plano en la banda de paso y tiene rizado en la de supresión, donde alcanza exactamente $1/A$ en sus máximos; tiene ceros sobre el eje $j\omega$. Para el tipo I, la condición en ω_s es $1 + \varepsilon^2 \cosh^2\left(N \operatorname{arccosh}(\omega_s/\omega_p)\right) \ge A^2$, que da

Propiedad 2.5: Orden de un Chebyshev

$$N \ge \frac{\operatorname{arccosh}(1/k_1)}{\operatorname{arccosh}(1/k)} = \frac{\operatorname{arccosh}\left(\sqrt{A^2-1}/\varepsilon\right)}{\operatorname{arccosh}(\omega_s/\omega_p)},$$

la misma para los dos tipos.

Con las especificaciones comunes, $1/k_1 = 196{,}5$ y $N \ge \operatorname{arccosh}(196{,}5)/\operatorname{arccosh}(5) = 5{,}97/2{,}29 = 2{,}61$, de modo que $N = 3$: uno menos que Butterworth. En Matlab y en `scipy.signal`, `cheby1(N,Rp,Wp)` recibe el rizado de la banda de paso y su borde, y `cheby2(N,Rs,Ws)` la atenuación de la banda de supresión y el suyo.

Erratas de versiones anteriores

En el resumen de órdenes de Matlab de las transparencias antiguas aparecía `cheby2(N,ap,wc,...)`; el segundo argumento de `cheby2` es la atenuación de la banda de supresión, `Rs`. La expresión del tipo II con $\cos^2(N \arccos 1)$ en el denominador tampoco era

correcta; la forma habitual es la de la definición 2.4.

2.4 Elíptico

El filtro *elíptico* o de Cauer usa una función racional de Chebyshev R_N, construida con funciones elípticas de Jacobi:

$$|H(j\omega)|^2 = \frac{1}{1+\varepsilon^2 R_N^2(\omega/\omega_p)}.$$

Tiene rizado en las dos bandas y ceros en el eje $j\omega$, y es el filtro de *menor orden* que cumple unas especificaciones dadas. El orden exacto es

$$N \geq \frac{K(k)\,K\!\left(\sqrt{1-k_1^2}\right)}{K(k_1)\,K\!\left(\sqrt{1-k^2}\right)},$$

con K la integral elíptica completa de primera especie. Una aproximación muy precisa es

$$k' = \sqrt{1-k^2}, \quad \rho_0 = \frac{1-\sqrt{k'}}{2(1+\sqrt{k'})}, \quad \rho = \rho_0 + 2\rho_0^5 + 15\rho_0^9 + 150\rho_0^{13}, \quad N \geq \frac{2\log_{10}(4/k_1)}{\log_{10}(1/\rho)}.$$

Para las especificaciones comunes, $k = \omega_p/\omega_s = 0{,}2$, $k' = 0{,}9798$, $\rho = 0{,}002551$ y $N \geq 2{,}23$, así que $N = 3$, el mismo orden que Chebyshev pero con más margen.

Erratas de versiones anteriores

El ejemplo elíptico antiguo escribía $k = \omega_s/\omega_p = 1/5$: el valor $1/5$ es correcto, pero corresponde a $k = \omega_p/\omega_s$. Además, la llamada `ellip(N,as,ap,wp,...)` tenía los dos primeros parámetros intercambiados: es `ellip(N,Rp,Rs,Wp)`.

2.5 Comparación

La figura 7 muestra las cuatro familias con el mismo orden, y la figura 8 sus polos y ceros. Cuanto más selectivo es un filtro, menos lineal es su fase: Butterworth es el de fase más lineal y el elíptico el de menos. La figura 9 compara los diseños de orden mínimo para la plantilla común.

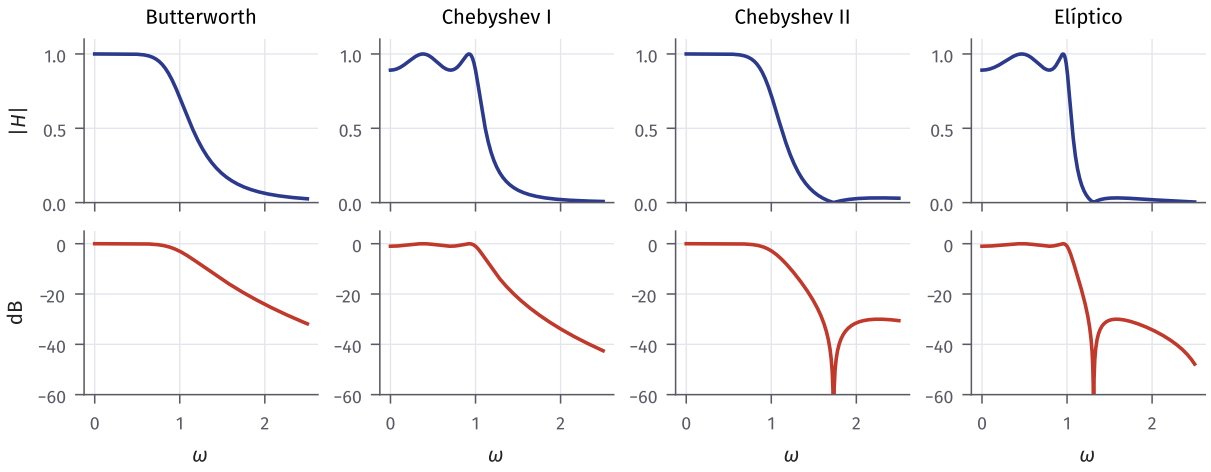


Figura 7. Butterworth, Chebyshev I y II y elíptico de orden 4, en escala lineal y en dB.

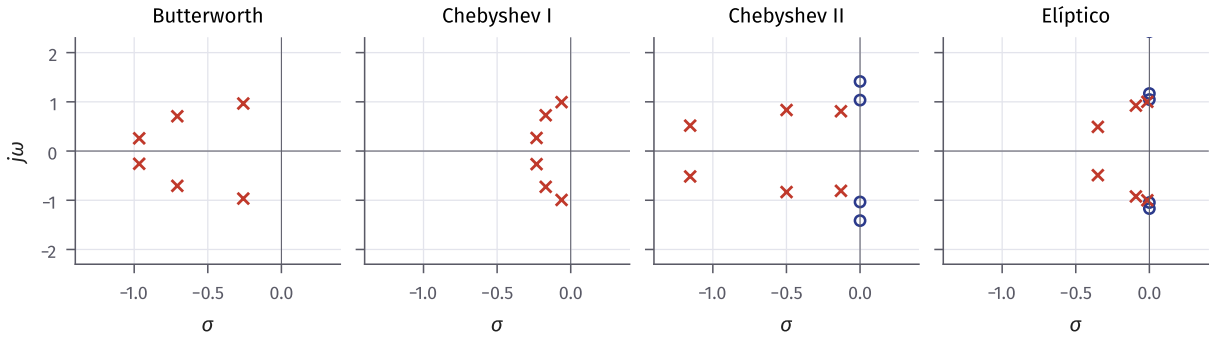


Figura 8. Polos (×) y ceros (∘) de las cuatro familias para $N = 6$.

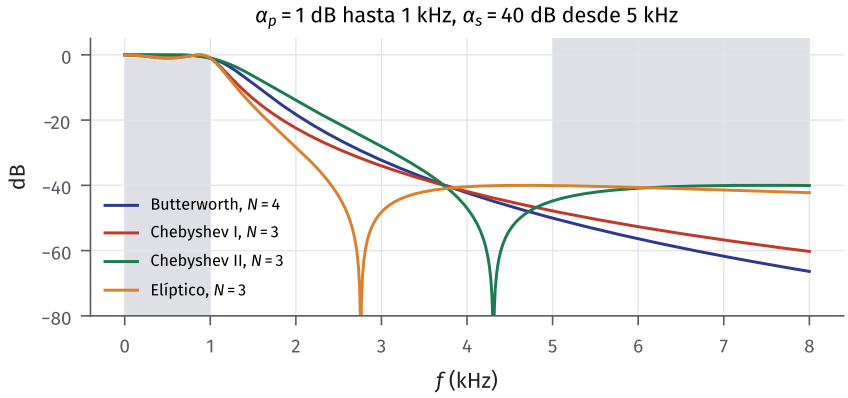


Figura 9. Diseños de orden mínimo para la plantilla común.

En código

Orden mínimo de cada familia y llamadas de diseño, con el orden correcto de los argumentos.

Python

```python
import numpy as np
from scipy import signal

wp = 2*np.pi*1e3; ws = 2*np.pi*5e3
ap, as_ = 1, 40
for f in (signal.buttord, signal.cheb1ord,
          signal.cheb2ord, signal.ellipord):
    N, wn = f(wp, ws, ap, as_, analog=True)
    print(f.__name__, N)   # 4, 3, 3, 3
b, a = signal.cheby1(3, ap, wp, analog=True)
b, a = signal.cheby2(3, as_, ws,
                     analog=True)
b, a = signal.ellip(3, ap, as_, wp,
                    analog=True)
```

Matlab

```matlab
wp = 2*pi*1e3; ws = 2*pi*5e3;
ap = 1; as = 40;
N1 = buttord(wp, ws, ap, as, 's')   % 4
N2 = cheb1ord(wp, ws, ap, as, 's')  % 3
N3 = cheb2ord(wp, ws, ap, as, 's')  % 3
N4 = ellipord(wp, ws, ap, as, 's')  % 3
% Rizado de la banda de paso (ap) antes
% que la atenuación (as):
[b, a] = cheby1(3, ap, wp, 's');
[b, a] = cheby2(3, as, ws, 's');
[b, a] = ellip(3, ap, as, wp, 's');
```

2.6 Transformaciones de frecuencia

Paso alto, paso banda y supresor se obtienen a partir de un prototipo paso bajo $H_{\mathrm{LP}}(s)$ con $\omega_p = 1$ mediante una sustitución $s = F(\hat{s})$, donde $\hat{s}$ y $\hat{\omega}$ son la variable y la frecuencia del filtro de destino (cuadro 2). Las transformaciones solo mueven las frecuencias: conservan los rizados y las atenuaciones.

Cuadro 2. Transformaciones de frecuencia desde un prototipo con $\omega_p = 1$. En paso banda y supresor, $\hat{\omega}_0^2 = \hat{\omega}_{p1}\hat{\omega}_{p2}$ y $B = \hat{\omega}_{p2} - \hat{\omega}_{p1}$.

Destino	Sustitución	Relación de frecuencias	Matlab / `scipy.signal`
Paso bajo	$s = \hat{s}/\hat{\omega}_p$	$\omega = \hat{\omega}/\hat{\omega}_p$	`lp2lp`
Paso alto	$s = \hat{\omega}_p/\hat{s}$	$\omega = -\hat{\omega}_p/\hat{\omega}$	`lp2hp`
Paso banda	$s = \dfrac{\hat{s}^2 + \hat{\omega}_0^2}{B\,\hat{s}}$	$\omega = \dfrac{\hat{\omega}^2 - \hat{\omega}_0^2}{B\,\hat{\omega}}$	`lp2bp`
Supresor	$s = \dfrac{B\,\hat{s}}{\hat{s}^2 + \hat{\omega}_0^2}$	$\omega = \dfrac{B\,\hat{\omega}}{\hat{\omega}_0^2 - \hat{\omega}^2}$	`lp2bs`

El procedimiento es:

1. llevar los bordes de supresión al prototipo con la relación de frecuencias y quedarse con el más exigente, $\omega_s = \min|\omega(\hat{\omega}_s)|$;
2. diseñar el prototipo con $\omega_p = 1$, ese ω_s, α_p y α_s;
3. sustituir: $H(\hat{s}) = H_{\mathrm{LP}}(F(\hat{s}))$.

El paso banda que resulta es geométricamente simétrico respecto a $\hat{\omega}_0$: dos frecuencias con la misma atenuación cumplen $\hat{\omega}_1\hat{\omega}_2 = \hat{\omega}_0^2$. Si los bordes de supresión dados no lo son, uno de los dos se cumple con margen; por eso se toma el mínimo. El orden del paso banda y del supresor es el doble del del prototipo (figura 10).

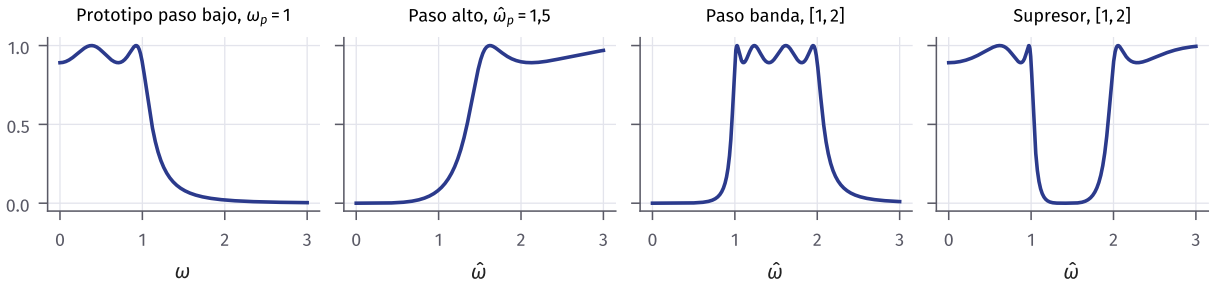


Figura 10. Un prototipo de Chebyshev I de orden 4 y los filtros que se obtienen con cada transformación.

Ejemplo 2.6: Paso alto de Butterworth

Banda de paso desde 4 kHz con rizado de 0,1 dB y atenuación mínima de 40 dB por debajo de 1 kHz. El prototipo tiene $\omega_p = 1$ y $\omega_s = \hat{\omega}_p/\hat{\omega}_s = 4$ (el signo de $\omega = -\hat{\omega}_p/\hat{\omega}$ no importa, porque $|H|$ es par). Con $E^2 - 1 = 10^{0{,}01} - 1 = 0{,}0233$,

$$N \geq \frac{\log_{10}(9999/0{,}0233)}{2\log_{10} 4} = 4{,}68 \;\rightarrow\; N = 5,$$

y el paso alto es $H(\hat{s}) = H_{\mathrm{LP}}(2\pi \cdot 4000/\hat{s})$.

En código

El ejemplo 2.6 con el prototipo y `lp2hp`, y en un solo paso con `butter(N,wn,'high','s')`.

Python

```python
import numpy as np
from scipy import signal

# Paso 4 kHz, 0,1 dB; supresión 1 kHz, 40 dB
wp_ = 2*np.pi*4e3; ws_ = 2*np.pi*1e3
# 1) Prototipo paso bajo con wp = 1
ws = wp_ / ws_                          # = 4
N, wc = signal.buttord(1, ws, 0.1, 40,
                       analog=True)
b, a = signal.butter(N, wc, analog=True)
# 2) Paso bajo -> paso alto: s = wp_/s
bh, ah = signal.lp2hp(b, a, wp_)
# Equivale a hacerlo en un solo paso:
N2, wn = signal.buttord(wp_, ws_, 0.1, 40,
                        analog=True)
b2, a2 = signal.butter(N2, wn, "highpass",
                       analog=True)
print(N, N2)                            # 5, 5
```

Matlab

```matlab
% Paso 4 kHz, 0,1 dB; supresión 1 kHz, 40 dB
wp_ = 2*pi*4e3; ws_ = 2*pi*1e3;
% 1) Prototipo paso bajo con wp = 1
ws = wp_ / ws_;                         % = 4
[N, wc] = buttord(1, ws, 0.1, 40, 's');
[b, a] = butter(N, wc, 's');
% 2) Paso bajo -> paso alto (Matlab):
%    [bh, ah] = lp2hp(b, a, wp_);
% Equivale a hacerlo en un solo paso:
[N2, wn] = buttord(wp_, ws_, 0.1, 40, 's');
[b2, a2] = butter(N2, wn, 'high', 's');
disp([N, N2])                           % 5, 5
```

Ejemplo 2.7: Paso banda elíptico y supresor de Chebyshev II

Paso banda: paso de 4 a 7 kHz con 1 dB; supresión por debajo de 3 kHz y por encima de 8 kHz con 22 dB. Trabajando en kHz (el factor 2π se simplifica), $\hat{\omega}_0^2 = 28$ y $B = 3$:

$$\omega(3) = \frac{|9-28|}{3\cdot 3} = 2{,}11, \qquad \omega(8) = \frac{64-28}{8\cdot 3} = 1{,}5,$$

así que $\omega_s = 1{,}5$ y un elíptico con $\alpha_p = 1$, $\alpha_s = 22$ necesita $N = 3$ (orden 6).
Supresor: paso por debajo de 4 y por encima de 7 kHz con 2 dB; supresión entre 5 y 6 kHz con 30 dB. De nuevo $\hat{\omega}_0^2 = 28$, $B = 3$:

$$\omega(5) = \frac{5\cdot 3}{28-25} = 5, \qquad \omega(6) = \frac{6\cdot 3}{36-28} = 2{,}25,$$

y $\omega_s = 2{,}25$. El orden de Chebyshev es $N \geq \operatorname{arccosh}(41{,}3)/\operatorname{arccosh}(2{,}25) = 3{,}02$, es decir, $N = 4$. `cheb2ord` aplicada a las especificaciones originales da $N = 3$, porque busca los bordes de paso que minimizan el orden: también es un diseño válido.

Erratas de versiones anteriores

Los ejemplos de paso banda y supresor antiguos decían «convertir el filtro paso bajo a paso alta» antes de usar `lp2bp` y `lp2bs`, hablaban de «atenuación superiora» (supresora) y de «rizo baso banda» (paso banda), y la frase «el filtro paso baja asociado al filtro paso baja» quería decir «el prototipo paso bajo asociado al filtro de destino». El ejemplo de paso banda antiguo llegaba a $\omega_s = 1{,}4$ estrechando un borde de paso para hacer la plantilla simétrica; el resultado es más exigente que el necesario, pero válido.

Ejercicios 4–7. *Herramienta interactiva:* `herramientas/server/T4_Filtros_Continuos.py`.

3 Filtros IIR

Un filtro IIR se diseña casi siempre a partir de uno continuo: se obtiene $H_a(s)$ con los métodos de la apartado 2 y se convierte en $H(z)$. La conversión debe llevar el eje $j\omega$ a la circunferencia unidad, para conservar la respuesta en frecuencia; el semiplano izquierdo al interior del círculo, para conservar la estabilidad; y dar un $H(z)$ racional.

3.1 Invarianza al impulso

La idea más directa es muestrear la respuesta al impulso: $h[n] = T\,h_a(nT)$ (el factor T mantiene la ganancia en la banda de paso). Si H_a tiene polos simples, por fracciones simples,

$$H_a(s) = \sum_k \frac{r_k}{s - p_k} \;\Rightarrow\; h_a(t) = \sum_k r_k \mathrm{e}^{p_k t} u(t) \;\Rightarrow\; H(z) = \sum_k \frac{T\,r_k}{1 - \mathrm{e}^{p_k T} z^{-1}}.$$

Cada polo p_k se convierte en $\mathrm{e}^{p_k T}$, que está dentro del círculo si $\operatorname{Re} p_k < 0$: la estabilidad se conserva. Pero, por el teorema del muestreo (Tema 2),

$$H(\mathrm{e}^{j\Omega}) = \sum_{m=-\infty}^{\infty} H_a\Big(j\frac{\Omega - 2\pi m}{T}\Big),$$

y como ningún filtro continuo de orden finito está limitado en banda, siempre hay *aliasing* (figura 11). Es tolerable si H_a es despreciable por encima de π/T, lo que ocurre con paso bajo y paso banda; con paso alto y supresor de banda el método no sirve. Se usa sobre todo cuando interesa reproducir la forma de $h_a(t)$, por ejemplo al simular un sistema continuo.

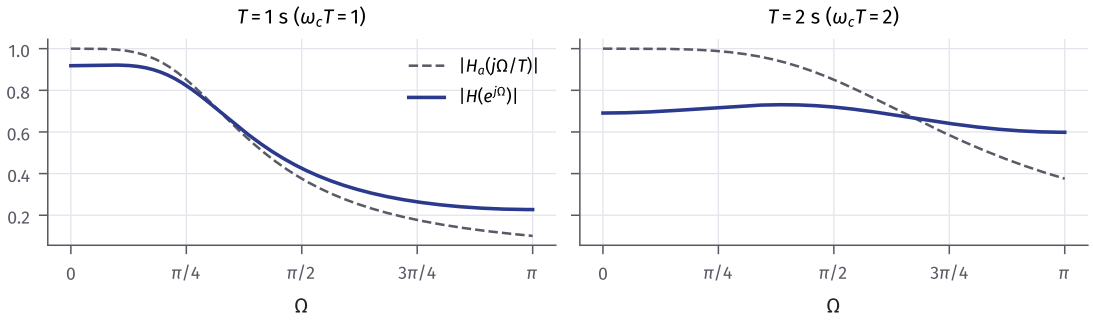


Figura 11. Invarianza al impulso de un Butterworth de orden 2 con $\omega_c = 1$ rad/s y dos periodos de muestreo. Con $T = 2$ s el aliasing deforma por completo la respuesta.

3.2 Transformada bilineal

Definición 3.1: Transformada bilineal

$$s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}} \qquad \Longleftrightarrow \qquad z = \frac{1 + sT/2}{1 - sT/2}, \qquad H(z) = H_a(s)\Big|_{s=\frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}}.$$

Aparece al integrar numéricamente con la regla del trapecio: $y[n] = y[n-1] + \frac{T}{2}(x[n] + x[n-1])$ tiene función de transferencia $\frac{T}{2}\frac{1+z^{-1}}{1-z^{-1}}$, que aproxima al integrador $1/s$. Con $s = \sigma + j\omega$, $|z|^2 = \frac{(1+\sigma T/2)^2 + (\omega T/2)^2}{(1-\sigma T/2)^2 + (\omega T/2)^2}$,

que es menor que 1 si y solo si $\sigma < 0$: el semiplano izquierdo va al *interior* del círculo y el eje $j\omega$ *sobre* la circunferencia. En $z = e^{j\Omega}$,

$$j\omega = \frac{2}{T}\,\frac{1 - e^{-j\Omega}}{1 + e^{-j\Omega}} = \frac{2}{T}\,\frac{e^{j\Omega/2} - e^{-j\Omega/2}}{e^{j\Omega/2} + e^{-j\Omega/2}} = j\,\frac{2}{T}\tan\frac{\Omega}{2}.$$

Propiedad 3.2: Relación de frecuencias de la bilineal

$$\omega = \frac{2}{T}\tan\frac{\Omega}{2}, \qquad \Omega = 2\arctan\frac{\omega T}{2}.$$

Todo el eje $\omega \in [0, \infty)$ se comprime en $\Omega \in [0, \pi)$, sin aliasing.

La compresión deforma las frecuencias (figura 12), pero conserva los niveles: un rizado de 1 dB del filtro continuo es un rizado de 1 dB del discreto. Para que los bordes del discreto queden en Ω_p y Ω_s, hay que *predistorsionar*: el filtro continuo se diseña con bordes $\omega_p = \frac{2}{T}\tan\frac{\Omega_p}{2}$ y $\omega_s = \frac{2}{T}\tan\frac{\Omega_s}{2}$. Como T aparece en la predistorsión y en la bilineal, se cancela y no tiene por qué ser el periodo de muestreo; lo habitual es tomar $T = 2$, con lo que $\omega = \tan(\Omega/2)$ y $s = \frac{1-z^{-1}}{1+z^{-1}}$. Los ceros en el infinito de un paso bajo continuo van a $\Omega = \pi$, es decir, a $z = -1$.

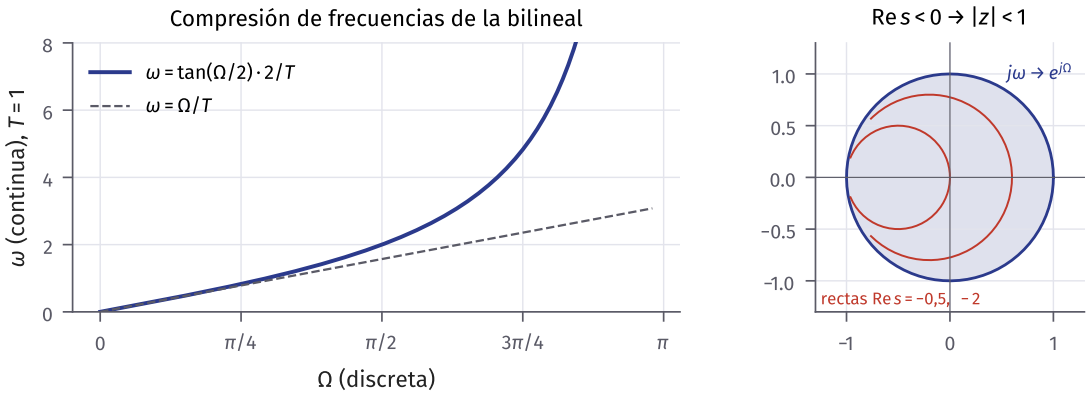


Figura 12. Relación de frecuencias de la bilineal ($T = 1$) y transformación del semiplano izquierdo en el interior del círculo unidad.

Erratas de versiones anteriores

Las transparencias antiguas decían que la bilineal lleva «el semiplano izquierdo al círculo unidad»: lo lleva al *interior*; es el eje $j\omega$ el que va a la circunferencia. Usaban ω_d y ω_a para las frecuencias discreta y continua; aquí son Ω y ω.

3.3 Procedimiento de diseño

1. Pasar los bordes a frecuencia discreta, $\Omega = 2\pi f / f_s$. Las atenuaciones no cambian.
2. Predistorsionar con $T = 2$: $\omega = \tan(\Omega/2)$ en cada borde.
3. Diseñar el filtro continuo: prototipo paso bajo y, si hace falta, transformación de frecuencias.
4. Aplicar la bilineal, $H(z) = H_a\big(\frac{1-z^{-1}}{1+z^{-1}}\big)$; con `bilinear` se indica $f_s = 1/T = 0{,}5$.
5. Comprobar la respuesta con `freqz`.

Ejemplo 3.3: Paso bajo de Butterworth

$f_s = 200\,\text{kHz}$, banda de paso hasta 25 kHz con rizado de 0,5 dB y atenuación mínima de 15 dB desde 55 kHz. Los bordes discretos son $\Omega_p = 2\pi \cdot 25/200 = 0{,}25\pi$ y $\Omega_s = 0{,}55\pi$, y predistorsionados $\omega_p = \tan(0{,}125\pi) = 0{,}4142$ y $\omega_s = \tan(0{,}275\pi) = 1{,}1708$. Con $E^2 - 1 = 0{,}1220$ y $A^2 - 1 = 30{,}62$,

$$N \geq \frac{\log_{10}(30{,}62/0{,}1220)}{2\log_{10}(1{,}1708/0{,}4142)} = 2{,}66 \;\rightarrow\; N = 3, \qquad \omega_c \in [0{,}588,\ 0{,}663].$$

Con $\omega_c = 0{,}588$ y la bilineal,

$$H(z) = \frac{0{,}0662\,(1 + z^{-1})^3}{1 - 0{,}9357z^{-1} + 0{,}5672z^{-2} - 0{,}1016z^{-3}},$$

que atenúa 18 dB a 55 kHz (figura 13).

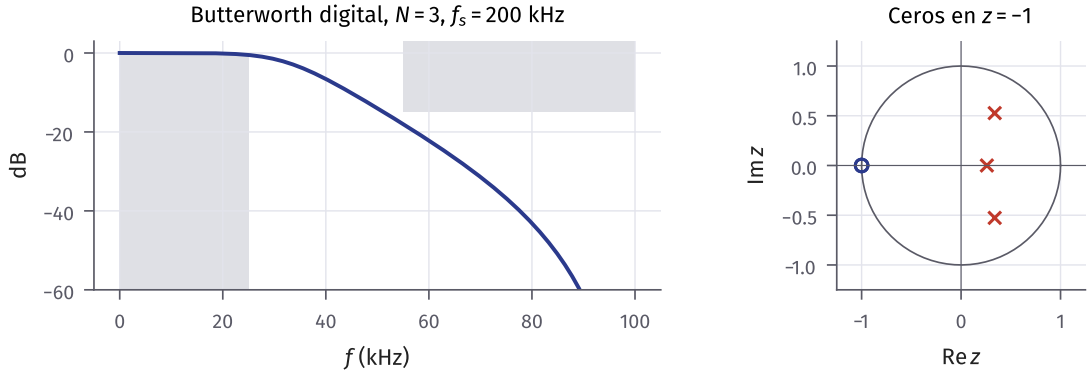


Figura 13. Respuesta y polos y ceros del ejemplo 3.3.

En código

El ejemplo 3.3 paso a paso (predistorsión, diseño continuo, bilineal) y con una sola llamada a `butter` con frecuencias discretas.

Python

```python
import numpy as np
from scipy import signal

fs = 200e3                      # Hz
Wp = 2*np.pi*25e3/fs            # π0,25
Ws = 2*np.pi*55e3/fs            # π0,55
# 1) Predistorsión con T = 2
wp, ws = np.tan(Wp/2), np.tan(Ws/2)
# 2) Filtro continuo
N, wc = signal.buttord(wp, ws, 0.5, 15,
                       analog=True)
b, a = signal.butter(N, wc, analog=True)
# 3) Bilineal: fs = 1/T = 0,5
bz, az = signal.bilinear(b, a, fs=0.5)
# Lo mismo en un paso (frecuencias en Hz):
N2, Wn = signal.buttord(25e3, 55e3, 0.5,
                        15, fs=fs)
bz2, az2 = signal.butter(N2, Wn, fs=fs)
print(N, np.allclose(az, az2))  # 3 True
```

Matlab

```matlab
fs = 200e3;                     % Hz
Wp = 2*pi*25e3/fs;              % π0,25
Ws = 2*pi*55e3/fs;              % π0,55
% 1) Predistorsión con T = 2
wp = tan(Wp/2); ws = tan(Ws/2);
% 2) Filtro continuo
[N, wc] = buttord(wp, ws, 0.5, 15, 's');
[b, a] = butter(N, wc, 's');
% 3) Bilineal: fs = 1/T = 0,5
[bz, az] = bilinear(b, a, 0.5);
% Lo mismo en un paso (frecuencias
% normalizadas a fs/2):
[N2, Wn] = buttord(Wp/pi, Ws/pi, 0.5, 15);
[bz2, az2] = butter(N2, Wn);
disp([N, N2])                   % 3, 3
```

Si se omite `'s'` (o `analog=True`), las funciones de orden y diseño de Matlab y `scipy.signal` hacen la predistorsión y la bilineal internamente; las frecuencias se dan normalizadas a la de Nyquist, $\Omega/\pi \in [0,1]$, o en hercios con el argumento `fs` de `scipy`. El tipo se indica con `'high'` y `'stop'` en Matlab, y con `'highpass'`, `'bandpass'` y `'bandstop'` en Python.

Erratas de versiones anteriores

Una transparencia antigua decía que, con $T = 2$, la orden `bilinear` se llama con tercer parámetro 0,5. Es correcto, pero la frase estaba incompleta («hacen de la orden bilineal se llamen»).

Ejemplo 3.4: Paso alto de Chebyshev I y paso banda de Butterworth

Paso alto, $f_s = 2$ kHz: paso desde 700 Hz con 1 dB, supresión por debajo de 500 Hz con 32 dB. $\Omega_p = 0{,}7\pi$ y $\Omega_s = 0{,}5\pi$ dan $\hat\omega_p = \tan(0{,}35\pi) = 1{,}9626$ y $\hat\omega_s = 1$. En el prototipo, $\omega_s = \hat\omega_p/\hat\omega_s = 1{,}9626$ y, con $\sqrt{A^2-1}/\varepsilon = 78{,}2$, $N \geq \operatorname{arccosh}(78{,}2)/\operatorname{arccosh}(1{,}9626) = 3{,}90$, es decir, $N = 4$.
Paso banda: paso en $[0{,}45\pi, 0{,}65\pi]$ con 1 dB y supresión fuera de $[0{,}3\pi, 0{,}75\pi]$ con 40 dB. Predistorsionando, $\hat\omega_{p1} = 0{,}8541$, $\hat\omega_{p2} = 1{,}6319$, $\hat\omega_{s1} = 0{,}5095$ y $\hat\omega_{s2} = 2{,}4142$; $\hat\omega_0^2 = 1{,}3937$ y $B = 0{,}7778$. Los bordes de supresión van a 2,86 y 2,36 en el prototipo, así que $\omega_s = 2{,}36$ y $N \geq 6{,}14$: $N = 7$, un filtro de orden 14 (figura 14).

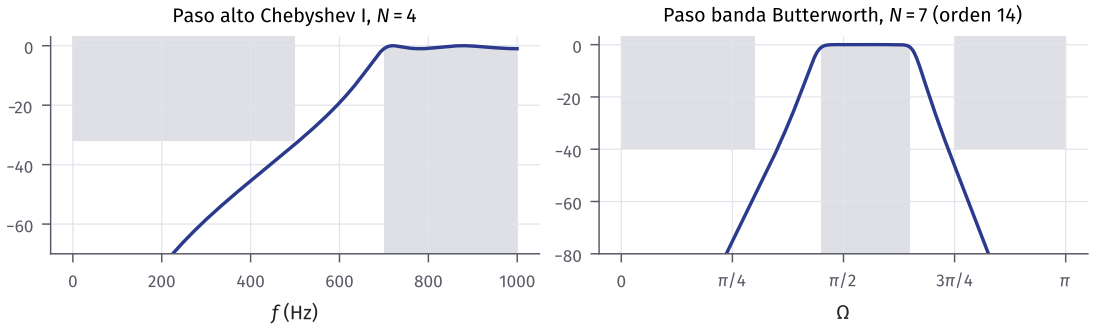


Figura 14. Filtros del ejemplo 3.4.

En código

Diseño directo del paso banda del ejemplo 3.4 en secciones de orden 2.

Python

```python
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt

# Bordes normalizados a π (1 = fs/2)
Wp = [0.45, 0.65]; Ws = [0.3, 0.75]
N, Wn = signal.buttord(Wp, Ws, 1, 40)
print(N)                    # 7: orden 14
sos = signal.butter(N, Wn, "bandpass",
                    output="sos")
W, H = signal.sosfreqz(sos, 2000)
plt.plot(W/np.pi, 20*np.log10(abs(H) + 1e-12))

plt.xlabel("Ωπ/"); plt.ylim(-80, 3)
plt.show()
# Filtrar una señal: signal.sosfilt(sos, x)
```

Matlab

```matlab
% Bordes normalizados a π (1 = fs/2)
Wp = [0.45 0.65]; Ws = [0.3 0.75];
[N, Wn] = buttord(Wp, Ws, 1, 40);
disp(N)                     % 7: orden 14
[b, a] = butter(N, Wn);     % paso banda
[H, W] = freqz(b, a, 2000);
plot(W/pi, 20*log10(abs(H)))
xlabel('\Omega/\pi'); ylim([-80 3])
% Para filtrar, mejor por secciones:
[z, p, k] = butter(N, Wn);
sos = zp2sos(z, p, k);      % sosfilt(sos, x)
```

3.4 Transformaciones de frecuencia en discreto

Alternativamente, se puede diseñar un paso bajo discreto con corte Ω_c y transformarlo sustituyendo z^{-1} por un filtro paso todo de $\hat{z}^{-1}$, que lleva la circunferencia unidad sobre sí misma y el interior al interior (Constantinides, 1970). Las dos más sencillas son

$$\text{a paso bajo con corte } \hat{\Omega}_c: \quad z^{-1} \to \frac{\hat{z}^{-1}-\alpha}{1-\alpha\hat{z}^{-1}}, \qquad \alpha = \frac{\sin\big((\Omega_c-\hat{\Omega}_c)/2\big)}{\sin\big((\Omega_c+\hat{\Omega}_c)/2\big)};$$

$$\text{a paso alto con corte } \hat{\Omega}_c: \quad z^{-1} \to -\frac{\hat{z}^{-1}+\alpha}{1+\alpha\hat{z}^{-1}}, \qquad \alpha = -\frac{\cos\big((\Omega_c+\hat{\Omega}_c)/2\big)}{\cos\big((\Omega_c-\hat{\Omega}_c)/2\big)}.$$

Hay transformaciones de orden 2 a paso banda y supresor (Oppenheim y Schafer, capítulo 7). En Matlab están en `iirlp2hp`, `iirlp2bp` e `iirlp2bs`, con frecuencias normalizadas en $[0, 1]$.

Erratas de versiones anteriores

La transparencia antigua daba para el paso alto $\lambda = -\cos\big((\omega_c-\hat{\omega}_c)/2\big)/\cos\big((\omega_c+\hat{\omega}_c)/2\big)$, con numerador y denominador intercambiados; en el supresor aparecía $\tan((\hat{\omega}_{c2}-\hat{\omega}_{c2})/2)$ en lugar de $\tan((\hat{\omega}_{c2}-\hat{\omega}_{c1})/2)$, y la función de Matlab `iirlp2sp` es `iirlp2bs`.

3.5 Implementación

Un IIR de orden alto en forma directa (los coeficientes a_k y b_k del polinomio completo) es muy sensible al redondeo: las raíces de un polinomio de grado alto cambian mucho con pequeñas perturbaciones de sus coeficientes, y en los filtros selectivos los polos están muy juntos y cerca de la circunferencia. Se implementa como una cascada de *secciones de orden 2* (`sos`), cada una con un par de polos y un par de ceros: `butter(..., output="sos")` y `sosfilt` en Python; `zp2sos` y `sosfilt` en Matlab. La cuadro 3 compara los dos métodos de conversión.

Cuadro 3. Invarianza al impulso y transformada bilineal.

	Invarianza al impulso	Bilineal
Relación	$z = \mathrm{e}^{sT}$ para los polos	$s = \frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}$
Frecuencias	Lineal, $\Omega = \omega T$	Comprimidas, $\omega = \frac{2}{T}\tan\frac{\Omega}{2}$
Aliasing	Sí	No
Tipos	Paso bajo y paso banda	Todos
Conserva	La forma de $h(t)$	Los niveles de $\lvert H\rvert$

Ejercicios 8–11. *Herramienta interactiva:* `herramientas/server/T4_Filtros_IIR_Discretizacion.py`.

4 Filtros FIR

Un FIR de N coeficientes (orden $N-1$) es

$$y[n] = \sum_{k=0}^{N-1} h[k]\,x[n-k], \qquad H(\mathrm{e}^{j\Omega}) = \sum_{n=0}^{N-1} h[n]\,\mathrm{e}^{-j\Omega n}.$$

Todos sus polos están en $z = 0$, así que siempre es estable. No hay un filtro continuo del que partir, y se diseña directamente en discreto. Su mayor ventaja es que puede tener fase exactamente lineal.

4.1 Fase lineal

Si $h[n]$ es real y simétrica respecto a su centro $M = \frac{N-1}{2}$, $h[n] = h[N-1-n]$, los términos se agrupan por parejas:

$$h[n]\mathrm{e}^{-j\Omega n} + h[N-1-n]\mathrm{e}^{-j\Omega(N-1-n)} = \mathrm{e}^{-j\Omega M}\, h[n]\left(\mathrm{e}^{j\Omega(M-n)} + \mathrm{e}^{-j\Omega(M-n)}\right) = \mathrm{e}^{-j\Omega M}\, 2h[n]\cos\left(\Omega(M-n)\right).$$

Sumando todas las parejas (y el término central si N es impar), $H(\mathrm{e}^{j\Omega}) = \mathrm{e}^{-j\Omega M}A(\Omega)$ con $A(\Omega)$ real: la fase es lineal salvo saltos de π donde A cambia de signo, y el retardo de grupo vale $M = (N-1)/2$ muestras. Con antisimetría, $h[n] = -h[N-1-n]$, cada pareja da $2j\,h[n]\sin(\Omega(M-n))$ y $H = j\mathrm{e}^{-j\Omega M}A(\Omega)$: la fase es lineal con un desfase de $\pi/2$. Hay cuatro tipos (cuadro 4).

Cuadro 4. Los cuatro tipos de FIR de fase lineal, con $M = (N-1)/2$.

Tipo	Simetría	N	$A(\Omega)$	Ceros forzados
I	par	impar	$h[M] + \sum_{n=0}^{M-1} 2h[n]\cos(\Omega(M-n))$	ninguno
II	par	par	$\sum_{n=0}^{N/2-1} 2h[n]\cos(\Omega(M-n))$	$z=-1$
III	impar	impar	$\sum_{n=0}^{M-1} 2h[n]\sin(\Omega(M-n))$, con $h[M]=0$	$z=1$ y $z=-1$
IV	impar	par	$\sum_{n=0}^{N/2-1} 2h[n]\sin(\Omega(M-n))$	$z=1$

Los ceros forzados se ven en $A(\Omega)$: en el tipo II, en $\Omega = \pi$ todos los cosenos $\cos(\pi(M-n))$ tienen argumento múltiplo impar de $\pi/2$ y se anulan; en los tipos III y IV los senos se anulan en $\Omega = 0$, y en el III también en $\Omega = \pi$. Por eso el tipo II no puede ser paso alto ni supresor, el III solo puede ser paso banda, y el IV sirve para paso alto, derivadores y transformadores de Hilbert. El tipo I sirve para todo.

En el dominio z, la simetría equivale a $H(z) = \pm z^{-(N-1)}H(z^{-1})$: si z_0 es un cero, $1/z_0$ también lo es, y como h es real, también z_0^* y $1/z_0^*$. Los ceros aparecen en cuartetos, o en parejas recíprocas sobre el eje real, o en parejas conjugadas sobre la circunferencia (figura 15).

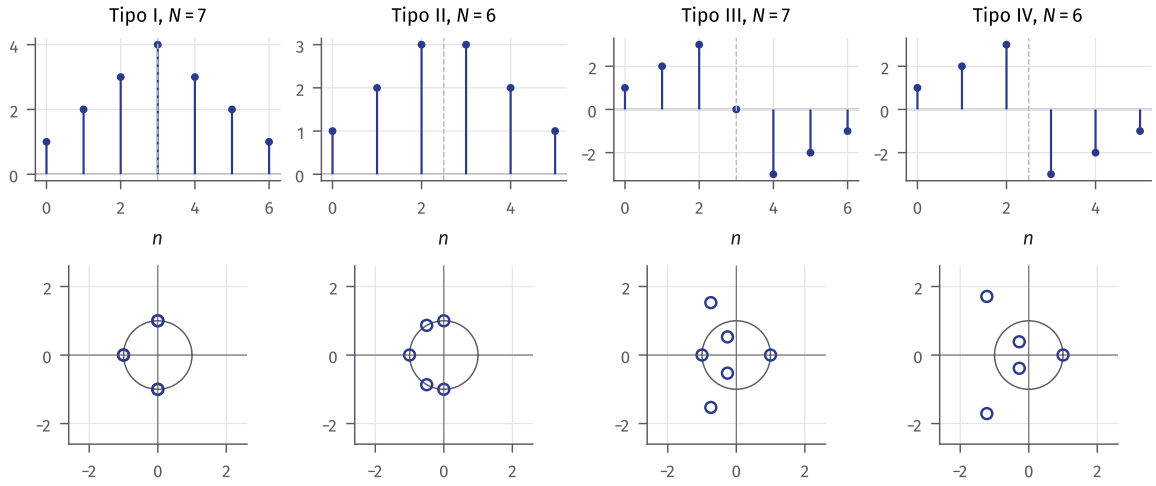


Figura 15. Un ejemplo de cada tipo y sus ceros. Obsérvense los ceros en $z = -1$ (tipos II y III) y en $z = 1$ (tipos III y IV).

Erratas de versiones anteriores

En la transparencia antigua de los ceros, los tipos aparecían con la paridad de N cambiada («Tipo 1 (N par)») y con números de ceros en ± 1 que no correspondían; la tabla resumen tenía el mismo

problema. Lo correcto es la cuadro 4: en $z = -1$ hay un número impar de ceros en los tipos II y III, y en $z = 1$ en los tipos III y IV. La nota a pie que decía que el desfase es $-\omega N/2$ se refiere a un filtro de $N + 1$ coeficientes.

4.2 Método de las ventanas

Se parte de la respuesta al impulso del filtro ideal con fase cero, $h_d[n] = \frac{1}{2\pi}\int_{-\pi}^{\pi} H_d(e^{j\Omega})e^{j\Omega n}\,d\Omega$ (cuadro 5). Como h_d es infinita y no causal:

1. se trunca a $|n| \le L$ ponderándola con una ventana $w[n]$: $h_w[n] = h_d[n]\,w[n]$, $N = 2L+1$ coeficientes;
2. se retrasa L muestras: $h[n] = h_w[n-L]$, $0 \le n \le 2L$ (figura 16).

h_d y w son pares, de modo que h es simétrica, de tipo I, con $H(e^{j\Omega}) = e^{-j\Omega L}H_w(e^{j\Omega})$. Por el teorema de la modulación,

$$H_w(e^{j\Omega}) = \frac{1}{2\pi}\int_{-\pi}^{\pi} H_d(e^{j\theta})\,W\left(e^{j(\Omega-\theta)}\right)d\theta\ :$$

el filtro ideal suavizado por la transformada de la ventana, igual que en el análisis espectral del Tema 3.

Cuadro 5. Respuestas al impulso de los filtros ideales con fase cero.

Filtro	$h_d[0]$	$h_d[n]$, $n \neq 0$
Paso bajo (Ω_c)	Ω_c/π	$\dfrac{\sin(\Omega_c n)}{\pi n}$
Paso alto (Ω_c)	$1-\Omega_c/\pi$	$-\dfrac{\sin(\Omega_c n)}{\pi n}$
Paso banda (Ω_1, Ω_2)	$(\Omega_2-\Omega_1)/\pi$	$\dfrac{\sin(\Omega_2 n)-\sin(\Omega_1 n)}{\pi n}$
Supresor (Ω_1, Ω_2)	$1-(\Omega_2-\Omega_1)/\pi$	$-\dfrac{\sin(\Omega_2 n)-\sin(\Omega_1 n)}{\pi n}$
Multibanda (A_k en $[\Omega_{k-1}, \Omega_k]$)	$\sum_k A_k\dfrac{\Omega_k-\Omega_{k-1}}{\pi}$	$\sum_k(A_k - A_{k+1})\dfrac{\sin(\Omega_k n)}{\pi n}$, con $A_{M+1}=0$

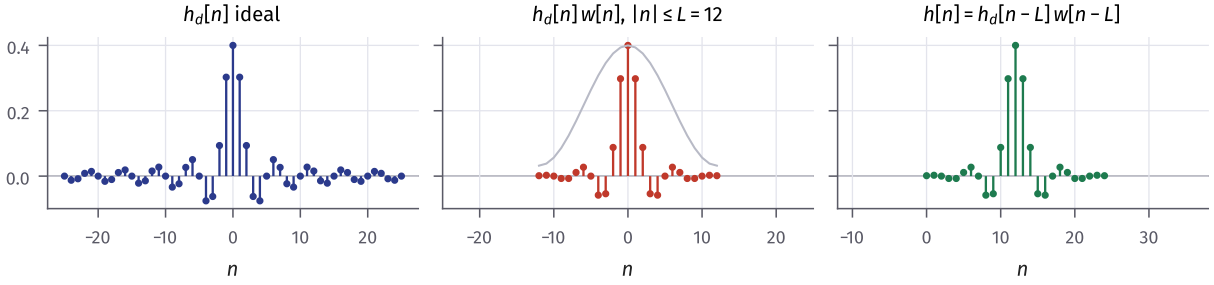


Figura 16. Método de las ventanas: respuesta ideal, truncada y ponderada (Hamming, $L = 12$), y retrasada.

Erratas de versiones anteriores

La expresión antigua del filtro multibanda ponderaba los senos con $A_k - A_{k-1}$; como $H = \sum_k A_k(H_{\mathrm{PB},\Omega_k} - H_{\mathrm{PB},\Omega_{k-1}})$, el coeficiente de $\sin(\Omega_k n)$ es $A_k - A_{k+1}$. En la diapositiva del procedimiento FIR, la ecuación $y[n] = \sum x[n-k]$ había perdido los coeficientes b_k, y se hablaba de «un FIR de orden $N+1$» con $N+1$ coeficientes (es de orden N).

4.3 Fenómeno de Gibbs y ventanas fijas

Con la ventana rectangular (truncamiento simple), el lóbulo principal de W, de anchura $4\pi/(2L+1)$, fija la anchura de la transición, que se estrecha al crecer L. Los lóbulos secundarios fijan el rizado, y como su área relativa no depende de L, el rizado tampoco: alrededor de un 9 % de sobreoscilación junto al borde y un primer lóbulo en la banda de supresión hacia -21 dB (figura 17). Es el *fenómeno de Gibbs*. Para reducir el rizado hay que usar ventanas que vayan suavemente a cero en los extremos (figura 18). Para $-L \le n \le L$:

$$w_{\text{Hann}} = \frac{1}{2} + \frac{1}{2}\cos\frac{2\pi n}{2L+1}, \qquad w_{\text{Hamming}} = 0{,}54 + 0{,}46\cos\frac{2\pi n}{2L+1},$$

$$w_{\text{Blackman}} = 0{,}42 + 0{,}5\cos\frac{2\pi n}{2L+1} + 0{,}08\cos\frac{4\pi n}{2L+1}.$$

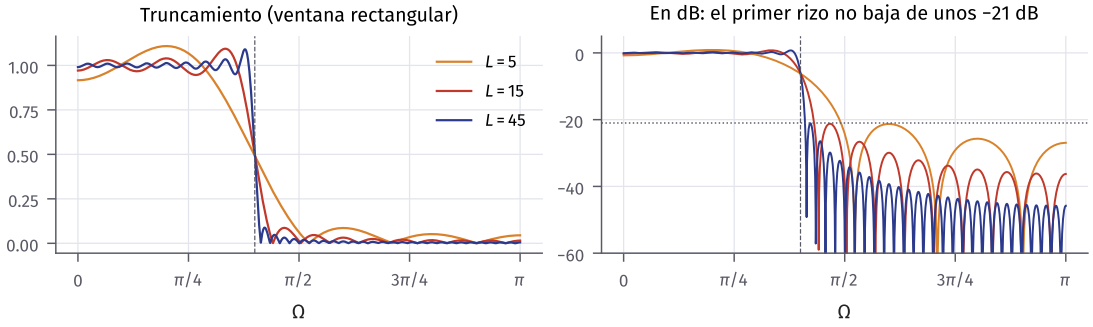


Figura 17. Truncamiento de un paso bajo ideal con $\Omega_c = 0{,}4\pi$: la transición se estrecha al crecer L, pero el rizado no baja.

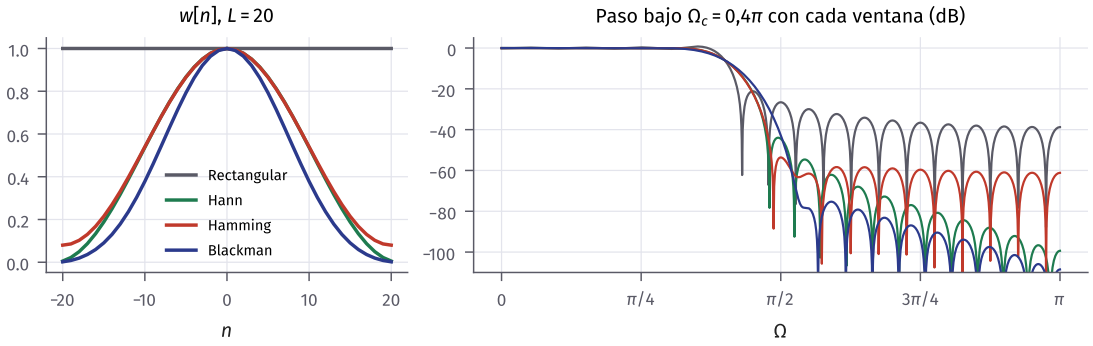


Figura 18. Ventanas fijas con $L = 20$ y el paso bajo que da cada una.

Cuadro 6. Ventanas fijas para diseño de filtros (valores de Mitra). La atenuación mínima es la del filtro, no la del lóbulo secundario de la ventana.

Ventana	Lóbulo principal	Lóbulo secundario	Atenuación mínima α_s	Transición $\Delta\Omega$
Rectangular	$4\pi/(2L+1)$	$-13{,}3$ dB	20,9 dB	$0{,}92\pi/L$
Hann	$8\pi/(2L+1)$	$-31{,}5$ dB	43,9 dB	$3{,}11\pi/L$
Hamming	$8\pi/(2L+1)$	$-42{,}7$ dB	54,5 dB	$3{,}32\pi/L$
Blackman	$12\pi/(2L+1)$	$-58{,}1$ dB	75,3 dB	$5{,}56\pi/L$

Con una ventana fija el rizado es el mismo en las dos bandas, $\delta_p \approx \delta_s$, y lo determina la ventana. El diseño se reduce a:

1. corte en el centro de la transición, $\Omega_c = (\Omega_p + \Omega_s)/2$, donde $|H| \approx 0{,}5$;

2. la ventana más sencilla con atenuación mínima mayor que α_s;
3. el menor L cuya transición sea menor que $\Delta\Omega = \Omega_s - \Omega_p$.

Ejemplo 4.1: Paso bajo con ventana fija

$\Omega_p = 0{,}3\pi$, $\Omega_s = 0{,}5\pi$, $\alpha_s = 40$ dB. $\Omega_c = 0{,}4\pi$ y $\Delta\Omega = 0{,}2\pi$. Sirven Hann, Hamming y Blackman; con Hann, $L \geq 3{,}11\pi/0{,}2\pi = 15{,}6$, así que $L = 16$ y $N = 33$ coeficientes; Hamming pediría $L = 17$ y Blackman $L = 28$. La atenuación conseguida es de 44 dB, con un retardo de 16 muestras.

Erratas de versiones anteriores

El ejemplo antiguo decía «filtros posibles son Hann, Hamming y Hamming»: el tercero es Blackman. En la fórmula de la ventana de Dolph–Chebyshev aparecía T_{2N} (es T_{2L}), y en la definición de T_l las dos ramas tenían la condición $|x| < 1$ (la segunda es $|x| > 1$).

En código

El ejemplo 4.1 construyendo $h[n]$ a mano y con `firwin`/`fir1`.

Python

```python
import numpy as np
from scipy import signal

# Ωp = π0,3, Ωs = π0,5, αs = 40 dB -> Hann
Wc = 0.4*np.pi              # Ω(p + Ωs)/2
L = 16                      # ππ3,11/(0,2)
n = np.arange(-L, L+1)
hd = Wc/np.pi * np.sinc(Wc/np.pi * n)
w = np.hanning(2*L + 3)[1:-1]
h = hd * w                  # 33 coef.
# h ya es causal si se indexa desde 0
W, H = signal.freqz(h, 1, 4096)
As = -20*np.log10(abs(H[W >= 0.5*np.pi]))
print(As.min())             # 44 dB
# Casi igual (extremos nulos):
h2 = signal.firwin(2*L + 1, 0.4,
                   window="hann")
```

Matlab

```matlab
% Ωp = π0,3, Ωs = π0,5, αs = 40 dB -> Hann
Wc = 0.4*pi;                % Ω(p + Ωs)/2
L = 16;                     % ππ3,11/(0,2)
n = -L:L;
hd = Wc/pi * sinc(Wc/pi * n);
w = hanning(2*L + 1).';     % sin ceros
h = hd .* w;                % 33 coef.
[H, W] = freqz(h, 1, 4096);
As = -20*log10(abs(H(W >= 0.5*pi)));
disp(min(As))               % 44 dB
% Casi igual (ganancia 1 en Ω = 0):
h2 = fir1(2*L, 0.4, hanning(2*L + 1));
```

4.4 Ventana de Kaiser

Las ventanas fijas no permiten elegir el rizado. La ventana de Kaiser tiene un parámetro β que ajusta el compromiso entre rizado y anchura de transición (figura 19):

$$w[n] = \frac{I_0\!\left(\beta\sqrt{1-(n/L)^2}\right)}{I_0(\beta)}, \quad |n| \leq L, \qquad I_0(u) = 1 + \sum_{r=1}^{\infty}\left(\frac{(u/2)^r}{r!}\right)^2,$$

donde I_0 es la función de Bessel modificada de orden cero. Kaiser obtuvo fórmulas empíricas para β y L:

Propiedad 4.2: Diseño con ventana de Kaiser

$$\beta = \begin{cases} 0{,}1102\,(\alpha_s - 8{,}7) & \alpha_s > 50 \\ 0{,}5842\,(\alpha_s - 21)^{0{,}4} + 0{,}07886\,(\alpha_s - 21) & 21 \le \alpha_s \le 50 \\ 0 & \alpha_s < 21 \end{cases}, \qquad 2L \ge \frac{\alpha_s - 8}{2{,}285\,\Delta\Omega}.$$

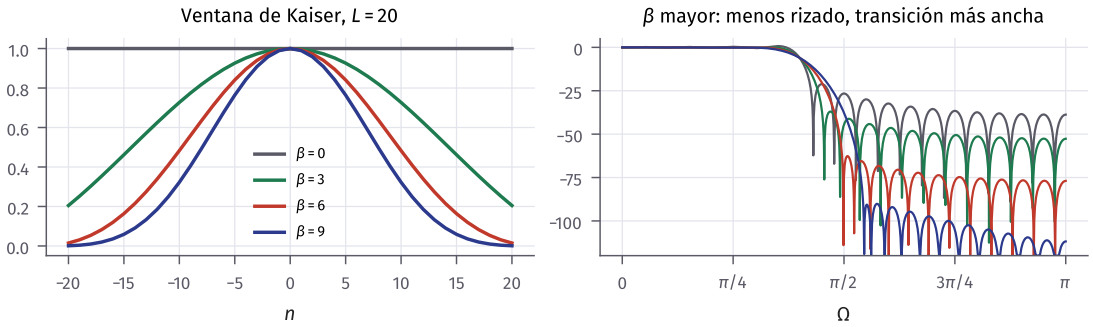


Figura 19. Ventana de Kaiser con $L = 20$ para varios valores de β, y el paso bajo correspondiente.

Para el ejemplo 4.1, $\beta = 0{,}5842 \cdot 19^{0{,}4} + 0{,}07886 \cdot 19 = 3{,}395$ y $2L \ge 32/(2{,}285 \cdot 0{,}2\pi) = 22{,}3$, de modo que $L = 12$: 25 coeficientes en lugar de 33, con 41,7 dB de atenuación (figura 20). Si se dan los rizados, $\alpha_s = -20\log_{10}\min(\delta_p, \delta_s)$, porque la ventana produce el mismo en las dos bandas. Otra ventana ajustable es la de *Dolph–Chebyshev* (`chebwin`), que pone todos los lóbulos secundarios a la misma altura; para ella $2L \approx (2{,}056\alpha_s - 16{,}4)/(2{,}285\Delta\Omega)$.

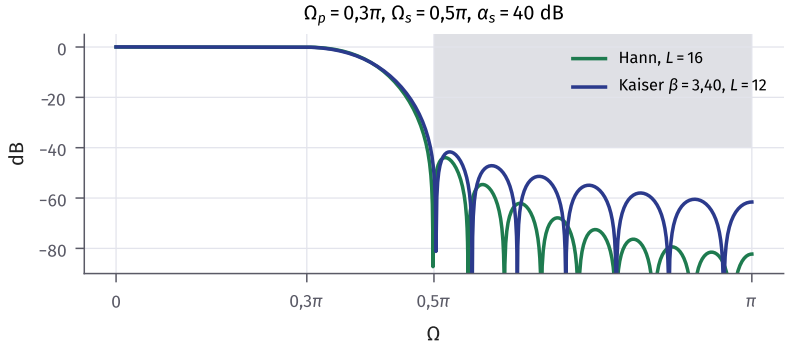


Figura 20. El paso bajo del ejemplo 4.1 con Hann ($L = 16$) y con Kaiser ($L = 12$).

Erratas de versiones anteriores

Las transparencias antiguas daban 0,5482 en lugar de 0,5842 en la fórmula de β, con lo que el ejemplo obtenía $\beta = 3{,}278$ en lugar de 3,395, y el intervalo intermedio aparecía como $50 \le \alpha_s \le 21$. El coeficiente del primer tramo es 0,1102, no 0,11102.

En código

Paso alto con $\Omega_s = 0{,}4\pi$, $\Omega_p = 0{,}55\pi$ y $\delta = 0{,}01$ con `kaiserord`, y el mismo filtro con Parks–McClellan. El paso alto debe ser de tipo I: si `kaiserord` propone un número par de coeficientes, se añade uno.

Python

```python
import numpy as np
from scipy import signal

# Paso alto: Ωs = π0,4, Ωp = π0,55,
# rizado δ = 0,01 en las dos bandas
A = -20*np.log10(0.01)          # 40 dB
N, beta = signal.kaiserord(A, 0.15)
N += (N % 2 == 0)               # impar
h = signal.firwin(N, 0.475, pass_zero=False,
                  window=("kaiser", beta))
print(N, round(beta, 3))        # 31, 3.395
# Parks-McClellan: rizado uniforme
hpm = signal.remez(N, [0, 0.4, 0.55, 1],
                   [0, 1], fs=2)
```

Matlab

```matlab
% Paso alto: Ωs = π0,4, Ωp = π0,55,
% rizado δ = 0,01 en las dos bandas
d = 0.01;
[N, Wn, beta, tipo] = kaiserord( ...
    [0.4 0.55], [0 1], [d d]);
N = N + mod(N, 2);              % orden par
h = fir1(N, Wn, tipo, kaiser(N+1, beta));
disp([N+1, beta])               % 31, 3.395
% Parks-McClellan: rizado uniforme
hpm = firpm(N, [0 0.4 0.55 1], [0 0 1 1]);
```

4.5 Parks–McClellan y muestreo en frecuencia

El método de Parks–McClellan busca el FIR de fase lineal que minimiza el *error máximo* ponderado entre $A(\Omega)$ y la respuesta ideal en las bandas especificadas. El teorema de alternancia asegura que la solución es *equirrizada*: el error alcanza su máximo con signos alternos en al menos $M+2$ frecuencias, y el algoritmo de Remez las encuentra. Con el mismo número de coeficientes, el rizado es menor que con cualquier ventana (figura 21), y se pueden pedir rizados distintos en cada banda. El orden se estima con la fórmula de Kaiser

$$N - 1 \approx \frac{-20\log_{10}\sqrt{\delta_p \delta_s} - 13}{14{,}6\,\Delta\Omega/2\pi}.$$

Se usa con `firpm` en Matlab y `remez` en `scipy.signal`.

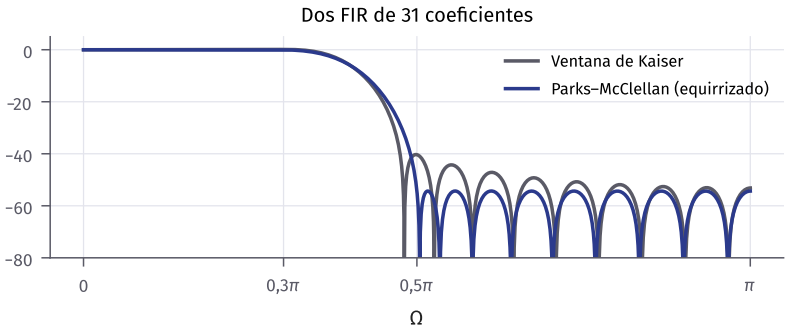


Figura 21. FIR de 31 coeficientes con Kaiser y con Parks–McClellan para las mismas bandas.

En el *muestreo en frecuencia* se fija la respuesta deseada en N frecuencias $\Omega_k = 2\pi k/N$ y se obtiene $h[n]$ con la IDFT, imponiendo la simetría de fase lineal. Entre las muestras, la respuesta interpola con rizado; se mejora dejando muestras de valor intermedio en la transición o aplicando una ventana a $h[n]$. `fir2` (Matlab) y `firwin2` (`scipy.signal`) hacen esto a partir de una respuesta definida a tramos, lo que permite filtros multibanda arbitrarios (figura 22).

4.6 Coste y retardo

Un FIR de N coeficientes cuesta N productos por muestra (la mitad aprovechando la simetría) y retrasa $(N-1)/2$ muestras. En audio a 44,1–48 kHz son habituales 50–200 coeficientes en filtros básicos, 100–

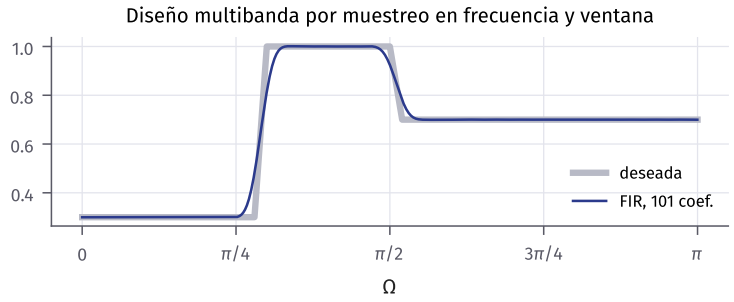


Figura 22. FIR multibanda de 101 coeficientes con `firwin2` y ventana de Hamming.

500 en filtros de cruce de altavoces, 500–2000 en cancelación de eco y hasta varios miles en procesado de alta calidad, con retardos de unos pocos a decenas de milisegundos. Para órdenes altos se filtra por bloques con la FFT (solapamiento y suma, Tema 3). En aplicaciones en tiempo real el retardo limita el orden; un IIR, o un FIR de fase mínima (apartado 5), retrasan mucho menos.

Ejercicios 12–15. *Herramienta interactiva:* `herramientas/server/T4_01_FIR_Ventana_Fija.py` y `T4_02_FIR_Kaiser.py`.

5 Filtros especiales

5.1 Fase cero

Un filtro de fase cero tiene $H(\mathrm{e}^{j\Omega})$ real y no negativa, lo que exige $h[n] = h[-n]$: no es causal. Dado cualquier $F(z)$, $H(z) = F(z)F(z^{-1})$ es de fase cero, porque $H(\mathrm{e}^{j\Omega}) = F(\mathrm{e}^{j\Omega})F^*(\mathrm{e}^{j\Omega}) = |F(\mathrm{e}^{j\Omega})|^2$ si f es real. Sus polos y ceros aparecen en parejas recíprocas, así que H no es causal y estable a la vez; pero si la señal está *grabada* se puede aplicar en dos pasos: se filtra x con F, se invierte la salida en el tiempo, se vuelve a filtrar y se invierte otra vez. Como invertir en el tiempo una señal real conjuga su DTFT,

$$Y = \big(F\,(F\,X)^*\big)^* = F^*F\,X = |F|^2 X.$$

La atenuación en dB se duplica y la fase es nula (figura 23). Es lo que hace `filtfilt` en Matlab y en `scipy.signal`, que además cuida los transitorios de los extremos.

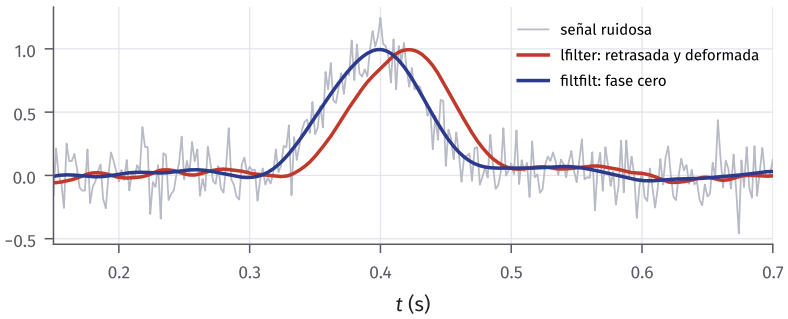


Figura 23. Un pulso con ruido filtrado con un Butterworth de orden 3 hacia delante (`lfilter`) y con fase cero (`filtfilt`).

En código

Filtrado normal y de fase cero; se compara la posición del máximo.

Python

```python
import numpy as np
from scipy import signal

fs = 500
t = np.arange(0, 1, 1/fs)
rng = np.random.default_rng(3)
x = np.exp(-((t - 0.4)/0.05)**2) \
    + 0.15*rng.standard_normal(t.size)
b, a = signal.butter(3, 15, fs=fs)
y1 = signal.lfilter(b, a, x)    # retrasa
y2 = signal.filtfilt(b, a, x)   # |H|^2
print(t[np.argmax(y1)], t[np.argmax(y2)])
```

Matlab

```matlab
fs = 500;
t = 0:1/fs:1-1/fs;
x = exp(-((t - 0.4)/0.05).^2) ...
    + 0.15*randn(size(t));
[b, a] = butter(3, 15/(fs/2));
y1 = filter(b, a, x);           % retrasa
y2 = filtfilt(b, a, x);         % |H|^2
[~, i1] = max(y1); [~, i2] = max(y2);
disp([t(i1), t(i2)])
```

5.2 Fase mínima, máxima y mixta

Reflejar un cero respecto a la circunferencia unidad, de z_0 a $1/z_0^*$, no cambia el módulo de la respuesta salvo una constante:

$$|1 - z_0 e^{-j\Omega}| = |z_0| \left|1 - \frac{1}{z_0^*} e^{-j\Omega}\right|.$$

Hay por tanto muchos sistemas con el mismo $|H|$ y distinta fase. Un sistema causal y estable es de *fase mínima* si todos sus ceros están dentro del círculo, de *fase máxima* si están todos fuera y de *fase mixta* en otro caso (figura 24). El de fase mínima tiene el menor retardo de grupo y la menor variación de fase de todos los que comparten módulo, concentra la energía de $h[n]$ al principio, y es el único cuya inversa $1/H(z)$ también es causal y estable, lo que importa para deshacer el efecto de un canal.

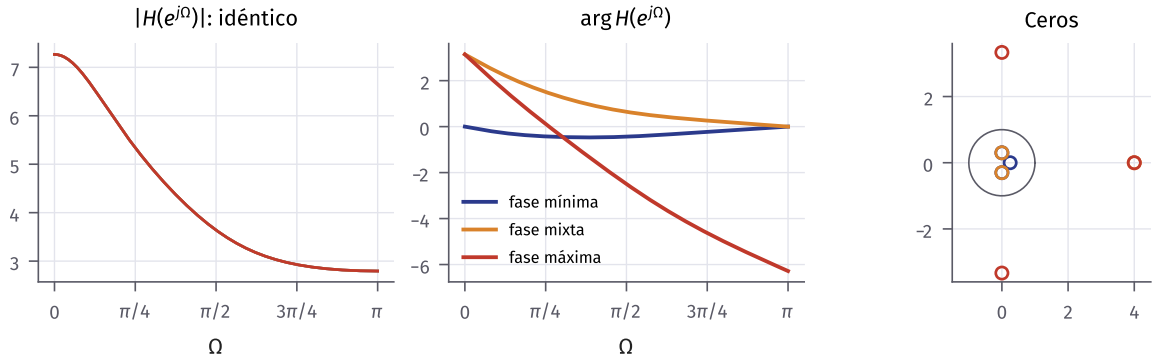


Figura 24. Tres sistemas con los mismos polos y el mismo módulo y ceros reflejados: fase mínima, mixta y máxima.

Erratas de versiones anteriores

Las transparencias antiguas definían H_{MAX} como un sistema «con todos sus ceros simétricos respecto al círculo unidad y el caso anterior»; quiere decir con los ceros de H_{MIN} reflejados, todos fuera del círculo. La variación de fase adicional del de fase máxima está entre 0 y $N\pi$ en $[0, \pi]$, con N el número de ceros reflejados.

5.3 Filtros paso todo

Un filtro *paso todo* tiene $|H(e^{j\Omega})| = 1$ para toda Ω. Con coeficientes reales y causal, tiene la forma

$$H_{PT}(z) = \frac{a_M + a_{M-1}z^{-1} + \cdots + z^{-M}}{1 + a_1 z^{-1} + \cdots + a_M z^{-M}} = \frac{z^{-M} D(z^{-1})}{D(z)},$$

con el numerador igual al denominador con los coeficientes en orden inverso. En la circunferencia, $|e^{-j\Omega M} D(e^{-j\Omega})| = |D^*(e^{j\Omega})| = |D(e^{j\Omega})|$, y el cociente tiene módulo 1. Cada polo p (dentro del círculo, si es estable) tiene enfrentado un cero en $1/p^*$. La fase decrece monótonamente $M\pi$ entre $\Omega = 0$ y $\Omega = \pi$, y el retardo de grupo es siempre positivo (figura 25). Se usan para corregir la fase de otro filtro sin tocar su módulo (igualadores de fase), como desfasadores y como bloques de filtros más complejos.

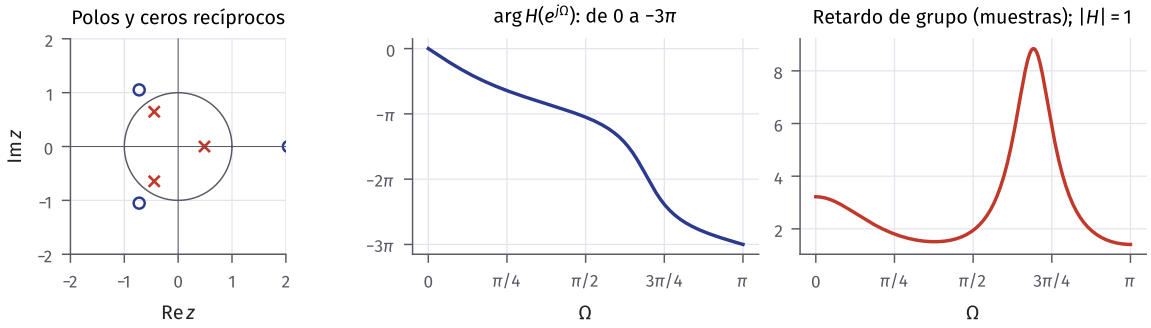


Figura 25. Paso todo de orden 3 con $D(z) = 1 + 0{,}4z^{-1} + 0{,}18z^{-2} - 0{,}3z^{-3}$: polos y ceros, fase y retardo de grupo.

Propiedad 5.1: Descomposición fase mínima por paso todo

Todo sistema racional, causal y estable se puede escribir $H(z) = H_{\min}(z)\, H_{PT}(z)$, con $H_{\min}$ de fase mínima y $|H_{\min}| = |H|$.

Basta reflejar hacia dentro cada cero z_0 con $|z_0| > 1$ y compensarlo con un paso todo de primer orden con el polo en $1/z_0^*$ y el cero en z_0. Por ejemplo,

$$\frac{1 + 3z^{-1}}{1 - 0{,}5z^{-1}} = \frac{3\,(1 + \frac{1}{3}z^{-1})}{1 - 0{,}5z^{-1}} \cdot \frac{\frac{1}{3} + z^{-1}}{1 + \frac{1}{3}z^{-1}}.$$

Como el paso todo siempre añade retardo, de aquí se deduce que el de fase mínima es el de menor retardo de grupo.

5.4 Filtros en peine

Sustituyendo z por z^L en un filtro $G(z)$ se obtiene un *filtro en peine*:

$$H(z) = G(z^L), \qquad H(e^{j\Omega}) = G(e^{j\Omega L}).$$

La respuesta se comprime en un factor L y se repite con periodo $2\pi/L$; en el tiempo, se intercalan $L - 1$ ceros entre los coeficientes de $g[n]$. Con $G(z) = \frac{1}{2}(1 + z^{-1})$, que tiene un cero en $\Omega = \pi$, sale $H(z) = \frac{1}{2}(1 + z^{-L})$, con ceros en $\Omega = (2k+1)\pi/L$ (figura 26).

Para eliminar una frecuencia f_0 y todos sus armónicos, como el zumbido de la red eléctrica, se parte de una muesca en continua, $G(z) = \frac{1+a}{2}\frac{1-z^{-1}}{1-az^{-1}}$, y se toma $L = f_s/f_0$:

$$H(z) = \frac{1 + a}{2}\,\frac{1 - z^{-L}}{1 - a\,z^{-L}}.$$

Sus ceros son las L raíces de la unidad, en $\Omega = 2\pi k/L$, es decir, en $f = kf_0$; los polos están en los mismos ángulos con radio $a^{1/L}$. Cuanto más cerca de 1 está a, más estrechas son las muescas y más largo el transitorio. Es el filtro que proporciona `iircomb` en Matlab y en `scipy.signal`.

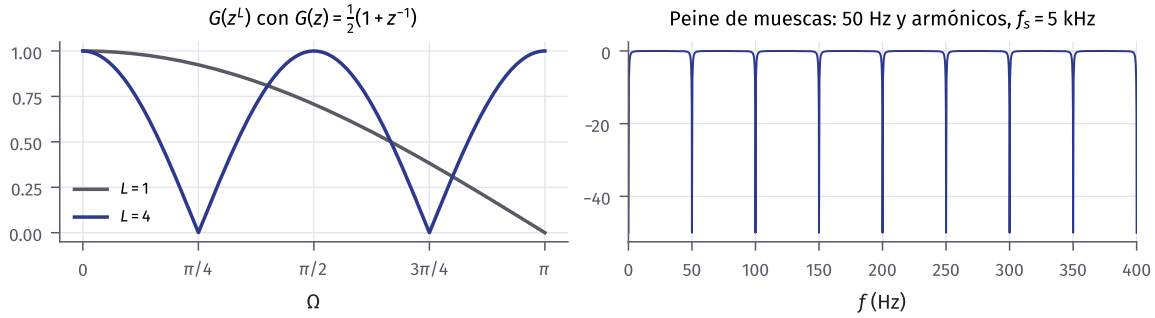


Figura 26. Peine FIR $\frac{1}{2}(1 + z^{-L})$ y peine de muescas para 50 Hz y sus armónicos con $f_s = 5$ kHz.

Erratas de versiones anteriores

El ejemplo antiguo de la red de 50 Hz con $f_s = 5$ kHz calculaba $L = f_s/f_0 = 50$; es $L = 5000/50 = 100$. El código asociado usaba además `iircomb(20,...)` y representaba la respuesta con $f_s = 300$ Hz, que no corresponden al enunciado.

En código

Peine de muescas para 50 Hz con $f_s = 5$ kHz y su respuesta en la componente útil (7 Hz), en 50 y 150 Hz y en 175 Hz.

Python

```python
import numpy as np
from scipy import signal

fs, f0 = 5000, 50
L = fs // f0                    # 100
a = 0.9                         # polos
# H(z) = (1+a)/2 (1 - z^-L)/(1 - a z^-L)
b = np.zeros(L + 1); b[[0, L]] = [1, -1]
d = np.zeros(L + 1); d[[0, L]] = [1, -a]
b *= (1 + a) / 2
f = np.array([7, 50, 150, 175])     # Hz
_, H = signal.freqz(b, d, f, fs=fs)
print(np.round(abs(H), 3))  # 1 0 0 1
```

Matlab

```matlab
fs = 5000; f0 = 50;
L = fs / f0;                    % 100
a = 0.9;                        % polos
% H(z) = (1+a)/2 (1 - z^-L)/(1 - a z^-L)
b = zeros(1, L+1); b([1 L+1]) = [1 -1];
d = zeros(1, L+1); d([1 L+1]) = [1 -a];
b = b * (1 + a) / 2;
f = [7 50 150 175];                 % Hz
H = freqz(b, d, f, fs);
disp(round(abs(H)*1000)/1000)  % 1 0 0 1
```

Ejercicios 16–18. *Herramienta interactiva:* `herramientas/server/T4_05_FIR_Fase_Cero.py`.

Bibliografía

- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos 5 (fase mínima, paso todo) y 10 (diseño de filtros).
- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Capítulos 5 (fase lineal, fase mínima y paso todo) y 7 (diseño de filtros).

- S. K. Mitra, *Procesamiento de señales digitales: un enfoque basado en computadora*, 3.ª ed. McGraw-Hill, 2007. Capítulos de diseño de filtros IIR y FIR; de ahí proceden los valores de la tabla de ventanas.
- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulos de filtros. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.
- V. K. Ingle y J. G. Proakis, *Digital Signal Processing Using MATLAB*, 3.ª ed. Cengage Learning, 2012. Capítulos 7 y 8, con ejemplos en Matlab.